1. Tại sao cần chiến lược Học Máy

Học Máy là nền tảng cho hàng loạt ứng dụng quan trọng như tìm kiếm trang web, lọc thư điện tử spam, nhận dạng giọng nói, gợi ý sản phẩm, và nhiều ứng dụng khác nữa. Nếu bạn cùng các thành viên trong nhóm đang làm một dự án học máy và rất muốn tiến triển nhanh chóng, thì quyển sách này là dành cho bạn.

Ví dụ: Xây dựng Startup về ảnh mèo

Giả sử bạn xây dựng công ty khởi nghiệp cung cấp không giới hạn ảnh mèo cho những người yêu thích.



Bạn dùng mạng nơ-ron cho hệ thống thị giác máy nhằm phát hiện mèo trong ảnh. Nhưng dở một cái là thuật toán bạn dùng chưa đủ độ chính xác. Bạn đang chịu rất nhiều áp lực để tăng chất lượng bộ phát hiện mèo. Bạn sẽ làm thế nào?

Nhóm bạn có thể đưa ra rất nhiều ý tưởng như:

- Lấy thêm dữ liệu: Sưu tầm thêm nhiều ảnh mèo.
- Lấy tập huấn luyện đa dạng hơn. Ví dụ như: ảnh mèo ở vị trí độc lạ, ảnh mèo với màu sắc khác thường, ảnh mèo được chụp với cấu hình máy ảnh khác nhau .v.v.
- Huấn luyện thuật toán lâu hơn bằng cách chạy thêm nhiều vòng lặp hạ gradient.
- Thử nghiệm mạng nơ-ron lớn hơn với nhiều tầng/nút ẩn/tham số hơn.
- Thử nghiệm mạng nơ-ron nhỏ hơn.
- Thử nghiệm kỹ thuật điều chuẩn -- *regularization* (ví dụ như điều chuẩn L2)
- Thay đổi kiến trúc mạng nơ-ron (ví dụ: hàm kích hoạt, số lượng nút ẩn, .v.v)
- ...

Nếu chọn đúng một trong những hướng kể trên, có thể bạn sẽ xây dựng nên một nền tảng ảnh mèo và startup thành công. Ngược lại, nếu chọn nhầm hướng, bạn có thể đánh mất cả tháng trời. Vậy phải làm như thế nào?

Cuốn sách này sẽ giúp bạn trả lời câu hỏi đó. Phần lớn các vấn đề về học máy đều có những dấu hiệu riêng ẩn chứa gợi ý về phương hướng giải quyết. Việc học để phát hiện ra những dấu hiệu đó sẽ giúp bạn tiết kiệm hàng tháng hay thậm chí hàng năm trời phát triển sản phẩm.

2. Cách sử dụng cuốn sách khi làm việc nhóm

Sau khi đọc xong cuốn sách này, bạn sẽ hiểu sâu hơn về cách lựa chọn hướng giải quyết kỹ thuật cho đề tài học máy.

Nhưng có thể cộng sự chưa rõ tại sao bạn lại chọn hướng đi như vậy. Ví dụ bạn muốn cả đội xác định và dùng một phép đo đơn trị, nhưng nếu mọi người không đồng tình, thì bạn sẽ làm gì để thuyết phục họ?

Đó là lý do tôi chủ tâm viết những chương rất ngắn. Bạn có thể dễ dàng thuyết phục quý đồng nghiệp bằng cách chia sẻ 1-2 trang của chương liên quan.

Chỉ với một vài thay đổi nhỏ về thứ tự ưu tiên có thể tác động lớn tới năng suất công việc của cả nhóm. Và bằng những thay đổi đó, tôi hi vọng bạn sẽ sớm trở thành siêu nhân Học Máy của cả đội!



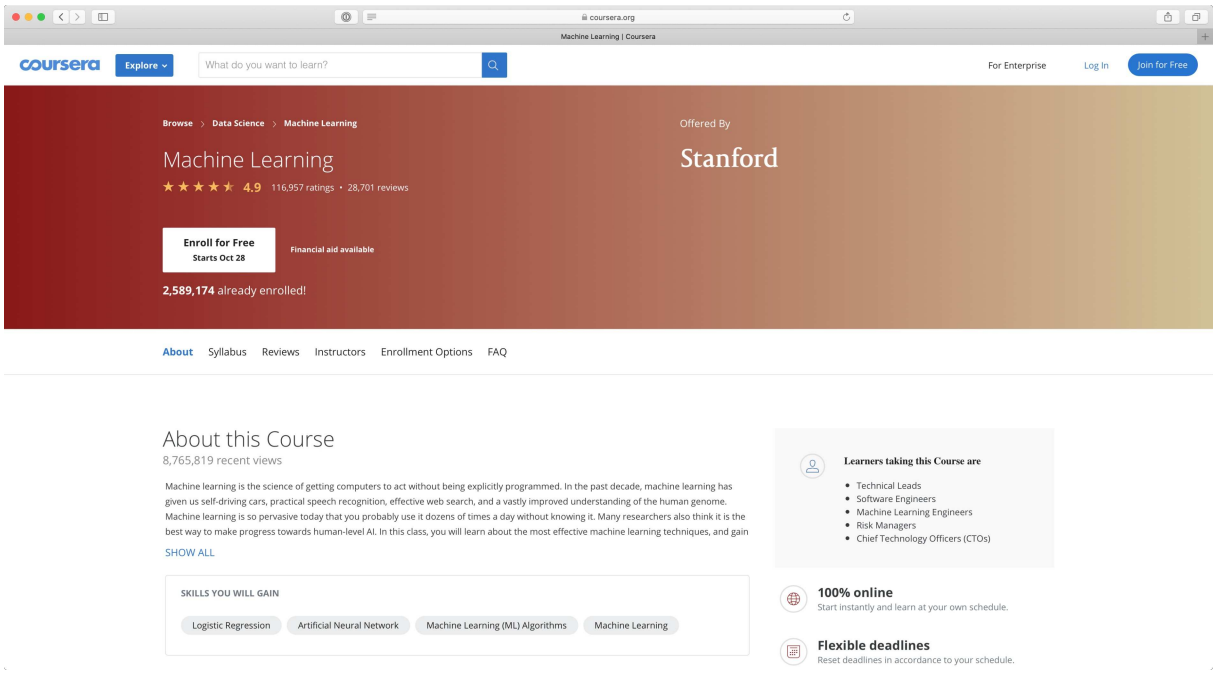
3. Kiến thức tiền đề và Ký hiệu

Nếu bạn đã từng học một lớp về Học Máy, ví dụ như lớp MOOC của tôi trên Coursera, hoặc bạn có kinh nghiệm áp dụng học có giám sát thì cuốn sách này sẽ dễ hiểu đối với bạn.

Tôi giả định rằng bạn đã quen thuộc với **học có giám sát**: học một hàm ánh xạ từ x tới y, sử dụng các cặp dữ liệu có nhãn (x,y). Các thuật toán học có giám sát bao gồm hồi quy tuyến tính, hồi quy logistic và mạng nơ-ron. Học Máy có rất nhiều dạng tuy nhiên phần lớn các giá trị thực tiễn của nó hiện nay đến từ học có giám sát.

Tôi sẽ thường xuyên đề cập đến mạng nơ-ron (còn được biết đến là "học sâu"). Bạn chỉ cần nắm được một số khái niệm cơ bản về mạng nơ-ron là có thể hiểu được nội dung cuốn sách.

Nếu những khái niệm nêu trên còn mới với bạn thì bạn hãy xem các video ba tuần đầu tiên của khóa học Machine Learning trên Coursera tại http://ml-class.org



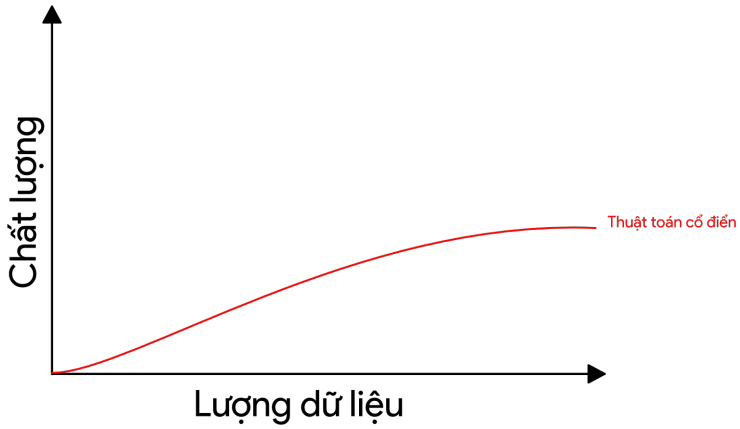
4. Quy mô là động lực phát triển học máy

Rất nhiều những ý tưởng của học sâu (mạng nơ-ron) đã xuất hiện từ hàng thập kỷ trước. Vậy tại sao tới bây giờ chúng mới bùng nổ như vậy?

Hai nguyên nhân chính là:

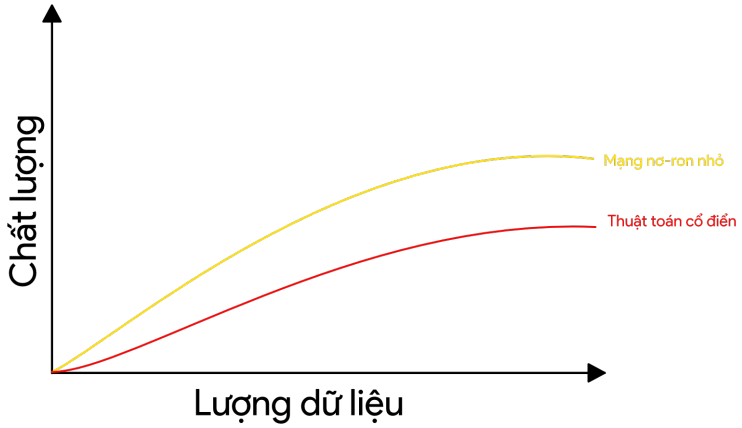
- **Sự sẵn có của dữ liệu.** Ngày nay, mọi người dành nhiều thời gian hơn bên những thiết bị số như máy tính xách tay, thiết bị di động, .v.v. Việc này tạo ra nguồn dữ liệu cực lớn dùng cho những thuật toán học máy.
- **Quy mô năng lực tính toán.** Chỉ tới một vài năm gần đây ta mới có thể huấn luyện mạng nơ-ron đủ lớn để tận dụng những bộ dữ liệu khổng lồ này.

Cho dù có thêm nhiều nhiều dữ liệu nữa, thường thì chất lượng của các thuật toán học máy cổ điển, như hồi quy logistic, cũng không tốt hơn. Nghĩa là đồ thị quá trình học chững lại và thuật toán ngừng cải thiện ngay cả khi có thêm dữ liệu:

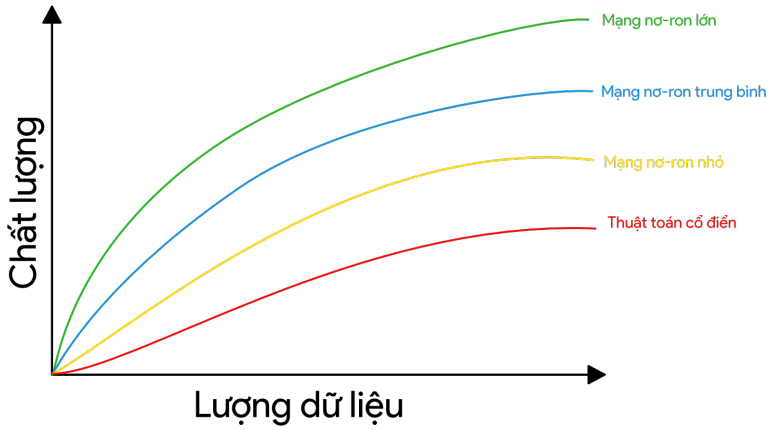


Như thể thuật toán cổ điển không biết xử lý thế nào với tất cả lượng dữ liệu ta đang có.

Nếu bạn huấn luyện một mạng nơ-ron nhỏ cho cùng một tác vụ học có giám sát, bạn có thể đạt chất lượng cao hơn một chút:



"Mạng neural nhỏ" ở đây có nghĩa là mạng nơ-ron với ít nút ẩn/tầng/tham số. Sau cùng, bạn có thể cải thiện chất lượng thêm nữa nếu dùng các mạng nơ-ron lớn hơn [1]:



Như vậy bạn đạt được chất lượng tốt nhất khi (i) huấn luyện mạng nơ-ron rất lớn -- tương ứng với đường chất lượng màu xanh lục và (ii) có lượng dữ liệu lớn.

Nhiều chi tiết khác như kiến trúc mạng nơ-ron cũng rất quan trọng, và có nhiều phát kiến trong lĩnh vực này. Tuy nhiên, một trong những cách đáng tin cậy hơn để tăng chất lượng thuật toán vẫn là (i) huấn luyện mạng lớn hơn và (ii) lấy thêm dữ liệu.

CHÚ THÍCH:

[1] Mặc dù hình vẽ thể hiện mạng nơ-ron cho kết quả tốt hơn với tập dữ liệu nhỏ, hiện tượng này ít nhất quán so với việc mạng nơ-ron hoạt động tốt với dữ liệu lớn. Với dữ liệu nhỏ, chất lượng thuật toán cổ điển có thể tốt hoặc kém hơn mạng nơ-ron và phụ thuộc vào các đặc trưng thủ công. Nếu ta chỉ có 20 mẫu huấn luyện thì việc dùng hồi quy logistic hay mạng nơ-ron không khác biệt nhiều; chọn khéo các đặc trưng thủ công sẽ giúp ích nhiều hơn so với việc chọn thuật toán. Còn nếu có một triệu mẫu, thì tôi sẽ chọn dùng mạng nơ-ron.

Để đạt được (i) và (ii) là một quá trình đặc biệt phức tạp. Vấn đề này sẽ được thảo luận đầy đủ và chi tiết trong cuốn sách này. Ta sẽ bắt đầu với các chiến lược thông thường và hữu ích cho cả thuật toán truyền thống lẫn mạng nơ-ron, từ đó hình thành các chiến lược tân tiến nhất để xây dựng các hệ thống học sâu.

Phần 1: Chuẩn bị tập phát triển và tập kiểm tra

5. Tập phát triển và tập kiểm tra

Trở lại với ví dụ bức ảnh mèo ở phần trước: khi bạn có một ứng dụng di động, và người dùng tải rất nhiều loại ảnh khác nhau lên ứng dụng của bạn. Bạn muốn tự động tìm ra đâu là các bức ảnh mèo.

Nhóm của bạn xây dựng một tập huấn luyện lớn bằng cách tải các bức ảnh mèo (các mẫu dương) và các bức ảnh không phải mèo (các mẫu âm) từ nhiều website khác nhau. Tập dữ liệu này sau đó được chia 70%/30% thành tập huấn luyện và tập kiểm tra. Sử dụng tập dữ liệu này, bạn tạo ra một bộ phát hiện mèo hoạt động tốt ở cả tập huấn luyện và tập kiểm tra.

Tuy nhiên, khi triển khai bộ phát hiện mèo này lên ứng dụng di động của bạn, bạn phát hiện ra chất lượng rất tệ!



Điều gì đã xảy ra?

Bạn nhận ra rằng các bức ảnh được người dùng tải lên có những tính chất khác so với những bức ảnh trên mạng mà bạn dùng để xây dựng tập huấn luyện: các bức ảnh được chụp bằng điện thoại có xu hướng có độ phân giải thấp hơn, bị nhòe (mờ) hoặc tối hơn. Do tập huấn luyện và tập kiểm tra của bạn được tạo lên từ ảnh trên mạng, nó không thể được sử dụng để khái quát tốt cho tính chất của phân phối mà bạn nhắm đến: ảnh chụp từ điện thoại.

Trước kỷ nguyên big data, có một nguyên tắc chung trong học máy là chia ngẫu nhiên tập huấn luyện và kiểm tra theo tỉ lệ 70%/30%. Tuy có thể dùng cách chia này, nhưng đây không phải là một ý hay vì ngày càng có nhiều ứng dụng với phân phối của tập huấn luyện (ảnh trên mạng trong ví dụ trên đây) khác phân phối mà bạn thực sự quan tâm (ảnh chụp từ điện thoại).

Chúng ta thường định nghĩa như sau:

- **Tập huấn luyện** — Là tập dữ liệu để chạy thuật toán học.
- **Tập phát triển** — Là tập dữ liệu được dùng để hiệu chỉnh các tham số, lựa chọn đặc trưng và quyết định các thay đổi liên quan đến thuật toán học. Đôi khi, nó còn được gọi là tập kiểm định chéo.
- **Tập kiểm tra** — Là tập dữ liệu dùng để đánh giá chất lượng của thuật toán học, nhưng không được dùng để quyết định các thay đổi liên quan đến thuật toán học hay các tham số.

Sau khi định nghĩa tập phát triển và tập kiểm tra, nhóm của bạn có thể thử nhiều ý tưởng khác nhau, ví dụ như các tham số khác nhau cho thuật toán học, để tìm ra ý tưởng tốt nhất. Tập phát triển và tập kiểm tra cho phép nhóm của bạn có thể đánh giá khả năng hoạt động của thuật toán một cách nhanh chóng.

Nói cách khác, **mục đích của tập phát triển và tập kiểm tra là hướng nhóm của bạn tới những thay đổi quan trọng nhất có thể làm để cải thiện trong hệ thống học máy**.

Vì vậy, bạn nên làm những điều sau đây:

Lựa chọn tập phát triển và tập kiểm tra để phản ánh đúng dữ liệu bạn sẽ gặp phải trong tương lai và muốn hoạt động tốt trên nó.

Nói cách khác, tập kiểm tra không nên chỉ đơn thuần là 30% dữ liệu hiện có, đặc biệt là khi dữ liệu tương lai (ảnh chụp từ điện thoại) về bản chất sẽ khác với dữ liệu trong tập huấn luyện (ảnh từ trên mạng).

Khi ứng dụng di động của bạn chưa được triển khai thì có thể bạn chưa có người dùng nào cả, nên việc có thể có được dữ liệu phản ánh chính xác dữ liệu tương lai là rất khó. Nhưng bạn vẫn có thể thử làm gần giống dữ liệu đó. Ví dụ, bạn có thể nhờ bạn bè chụp những bức ảnh mèo bằng điện thoại và gửi cho bạn. Một khi ứng dụng được triển khai, bạn có thể cập nhật tập phát triển/kiểm tra bằng dữ liệu người dùng thực tế.

Nếu bạn thực sự không có cách nào để có được dữ liệu gần giống với dữ liệu tương lai, có lẽ bạn có thể bắt đầu bằng việc sử dụng ảnh từ những trang web. Nhưng bạn nên nhận thức được nguy cơ việc này dẫn đến một hệ thống không có khả năng khái quát hóa tốt.

Vấn đề này đòi hỏi khả năng phán đoán để quyết định cần phải đầu tư bao nhiêu cho việc xây dựng tập phát triển và tập kiểm tra. Tuy nhiên đừng giả định phân phối của tập huấn luyện giống phân phối của tập kiểm tra. Hãy cố chọn ra những mẫu kiểm tra phản ánh điều mà bạn thực sự muốn thực hiện tốt, hơn là bất kì dữ liệu nào bạn tình cờ có được cho việc huấn luyện.

6. Tập phát triển và tập kiểm tra nên có cùng phân phối



Bạn có thể chia tập dữ liệu của ứng dụng ảnh mèo dựa theo bốn thị trường lớn nhất: (i) Hoa Kỳ, (ii) Trung Quốc, (iii) Ấn Độ, và (iv) Khu vực khác. Chúng ta có thể lấy dữ liệu từ Hoa Kỳ và Ấn Độ làm tập phát triển trong khi lấy Trung Quốc và Khu vực khác làm tập kiểm tra. Nói cách khác, liệu việc chọn ngẫu nhiên dữ liệu ảnh từ hai trong bốn khu vực trên làm tập phát triển và hai khu vực còn lại làm tập kiểm tra có đúng hay không?

Một khi định nghĩa được tập phát triển và tập kiểm tra, nhóm của bạn sẽ tập trung cải thiện chất lượng trên tập phát triển. Bởi vậy, tập phát triển cần phản ánh tác vụ bạn muốn cải thiện nhất đó là: hoạt động tốt trên không chỉ hai mà cả bốn thị trường.

Có một vấn đề nữa với việc tập phát triển và tập kiểm tra có phân phối khác nhau: Có khả năng nhóm của bạn sẽ xây dựng một thuật toán nào đó hoạt động tốt trên tập phát triển nhưng lại kém trên tập kiểm tra. Tôi đã từng thấy việc này dẫn đến những hệ quả gây thất vọng và lãng phí công sức. Hãy cố gắng tránh để điều này xảy ra.

Ví dụ, nhóm của bạn phát triển một hệ thống hoạt động tốt trên tập phát triển nhưng kém trên tập kiểm tra. Nếu tập phát triển và tập kiểm tra có cùng một phân phối, thì bạn có thể xác định ngay vấn đề: Mô hình của bạn đã quá khớp (*overfit*) tập phát triển. Cách xử lý hiển nhiên nhất đó là bổ sung thêm dữ liệu cho tập phát triển.

Nhưng nếu tập phát triển và tập kiểm tra có phân phối khác nhau, thì việc xác định vấn đề sẽ phức tạp hơn. Rất nhiều vấn đề có thể xảy ra như:

1. Bạn đã quá khớp tập phát triển.
2. Tập kiểm tra khó hơn tập phát triển. Vì thế thuật toán của bạn có thể đã hoạt động tốt hết mức có thể và không thể có thiện thêm nhiều nữa.
3. Tập kiểm tra không nhất thiết khó hơn, nhưng lại khác biệt so với tập phát triển. Do đó, việc thuật toán hoạt động tốt trên tập phát triển và kém trên tập kiểm tra là dễ hiểu. Trong trường hợp này, bỏ quá nhiều công sức cố gắng cải thiện chất lượng trên tập phát triển có thể trở nên vô nghĩa.

Làm việc với các ứng dụng học máy vốn dĩ đã khó. Việc không nhất quán giữa tập phát triển và kiểm tra khiến bạn càng khó chắc chắn về "liệu cải thiện chất lượng trên tập phát triển có đồng nghĩa với tăng chất lượng trên tập kiểm tra hay không?". Việc không đồng nhất giữa tập phát triển và tập kiểm tra khiến việc xác định những kỹ thuật giúp cải tiến chất lượng khó khăn hơn từ đó khó xắp xếp thứ tự ưu tiên giữa các tác vụ.

Nếu bạn đang làm việc với một bài toán đánh giá xếp hạng của bên thứ ba, họ có thể đã chỉ định tập phát triển và tập kiểm tra có phân phối khác nhau. So với bài toán có tập phát triển và tập kiểm tra có cùng một phân phối, thì chất lượng thuật toán của bạn trên bài toán đánh giá xếp hạng ở trên phụ thuộc nhiều vào may mắn hơn là kỹ năng. Việc phát triển thuật toán học mà được huấn luyện trên một phân phối này mà có thể khái quát hóa tốt trên một phân phối khác là một chủ đề nghiên cứu quan trọng. Tuy nhiên, nếu mục tiêu của bạn là cải tiến một ứng dụng học máy cụ thể thay vì làm nghiên cứu, thì tôi khuyên bạn chọn tập phát triển và tập kiểm tra có cùng một phân phối. Điều này sẽ khiến nhóm của bạn làm việc hiệu quả hơn.

7. Tập phát triển/kiểm tra cần lớn đến mức nào?

Tập phát triển phải đủ lớn để nhận ra sự khác biệt giữa các thuật toán bạn đang thử nghiệm. Ví dụ, nếu bộ phân loại A có độ chính xác 90,0% và bộ phân loại B có độ chính xác 90,1%, thì một tập phát triển có 100 mẫu sẽ không thể phát hiện sự khác biệt 0,1% này. So với các bài toán học máy khác mà tôi đã thấy, một tập phát triển chỉ với 100 mẫu là nhỏ. Các tập phát triển thường có từ 1.000 tới 10.000 mẫu. Với 10.000 mẫu, nhiều khả năng bạn sẽ thấy được mức cải thiện 0,1%. [2]

Trong các ứng dụng quan trọng và đã đã đưa vào khai thác -- ví dụ như quảng cáo, tìm kiếm trên web và gợi ý sản phẩm -- tôi đã thấy nhiều nhóm rất muốn cải thiện chất lượng thuật toán dù chỉ là 0,01%, vì nó có ảnh hưởng trực tiếp đến lợi nhuận của công ty. Trong trường hợp này, tập phát triển có thể lớn hơn 10.000 mẫu rất nhiều để có thể phát hiện ra những cải tiến thậm chí nhỏ hơn.

Vậy còn kích thước của tập kiểm tra thì sao? Nó cần đủ lớn để mang lại độ tin cậy cao về chất lượng tổng thể của hệ thống. Một công thức thực nghiệm phổ biến là sử dụng 30% dữ liệu làm tập kiểm tra. Cách làm này hiệu quả với những tập dữ liệu với lượng mẫu khiêm tốn từ 100 tới 10.000 mẫu. Tuy nhiên, trong kỷ nguyên big data với những bài toán học máy đôi khi có nhiều hơn một tỷ mẫu, tỉ lệ dữ liệu giữa tập phất triển và tập kiểm tra đã giảm xuống đáng kể, mặc dù số lượng mẫu trong hai tập này vẫn tăng lên. Thực sự không cần có tập phát triển/kiểm tra lớn quá mức để đánh giá chất lượng của các thuật toán.

CHÚ THÍCH:

[2] Trên lý thuyết, ta cũng có thể kiểm tra xem một thay đổi trong thuật toán có tạo ra sự khác biệt đáng kể về mặt thống kê trên tập phát triển hay không. Trong thực tế, hầu hết mọi người đều không quan tâm đến điều này (trừ khi họ muốn công bố các các bài báo khoa học). Tôi thường thấy các bài kiểm định thống kê không mấy hữu ích trong việc đánh giá tiến độ phát triển.

8. Thiết lập một phép đo đơn trị làm mục tiêu tối ưu

Độ chính xác trong phân loại là ví dụ của **phép đo đơn trị** -- phép đo được biểu diễn bằng chỉ một con số. Khi chạy bộ phân loại trên một tập phát triển (hoặc tập kiểm tra), độ chính xác được tính bằng số mẫu được phân loại đúng trên tổng số mẫu trong tập đó. Theo phép đo này, nếu độ chính xác của bộ phân loại A là 97% và của bộ phân loại B là 90% thì ta kết luận rằng bộ phân loại A cho kết quả tốt hơn.

Ngược lại, Precision và Recall[3] không phải là một phép đo đơn trị: chúng đưa ra hai chỉ số để đánh giá bộ phân loại. Việc so sánh các thuật toán với nhau sẽ trở nên khó hơn với những phép đo đa trị -- những phép đo được biểu diễn bằng nhiều hơn một số. Giả sử thuật toán trả về kết quả như sau:

Ở đây, không bộ phân loại nào tốt hơn một cách rõ ràng, vì vậy dựa vào kết quả trên ta không thể ngay lập tức chọn ra một bộ phân loại tốt hơn.

Bộ Phân Loại	Precision	Recall
A	95%	90%
B	98%	85%

Trong quá trình phát triển, nhóm bạn sẽ thử rất nhiều ý tưởng liên quan đến cấu trúc thuật toán, tham số mô hình, lựa chọn các đặc trưng, v.v.. Việc có một **phép đo đơn trị** như độ chính xác sẽ giúp xếp hạng các mô mình dựa theo những chất lượng trả về qua phép đo đó, từ đó nhanh chóng quyết định mô hình nào hoạt động tốt nhất.

Nếu bạn thực sự quan tâm đến cả Precision lẫn Recall. Tôi gợi ý sử dụng một trong những cách tiêu chuẩn để kết hợp các chỉ số đó thành một chỉ số duy nhất. Ví dụ, có thể lấy giá trị trung bình của Precision và Recall rồi thu về một phép đo đơn trị. Hoặc thay vào đó, bạn có thể tính "chỉ số F1", một biến thể của trung bình cộng và hoạt động tốt hơn việc chỉ lấy giá trị trung bình.

Việc có một phép đo đơn trị sẽ giúp tăng tốc khả năng đưa ra quyết định của bạn khi bạn phải lựa chọn một trong số lượng lớn các bộ phân loại. Phép đo đơn trị đưa ra một thứ hạng ưu tiên rõ ràng giữa những thuật toán đó, tạo ra một đường hướng rõ ràng để phát triển.

Bộ Phân Loại	Precision	Recall	Chỉ số F1
A	95%	90%	92.4%
B	98%	85%	91.0%

Một ví dụ cuối cùng, giả sử bạn đang theo dõi riêng biệt độ chính xác của bộ phân loại mèo trong bốn thị trường trọng điểm: (i) Mỹ, (ii) Trung Quốc, (iii) Ấn Độ, và (iv) những nước khác. Bạn sẽ thu về bốn phép đo. Bằng cách lấy giá trị trung bình hoặc giá trị trung bình có trọng số của bốn chỉ số này, bạn sẽ thu được một phép đo đơn

trị. Tính toán giá trị trung bình hoặc giá trị trung bình có trọng số là một trong những cách phổ biến nhất để kết hợp nhiều phép đo thành một.

CHÚ THÍCH:

[3] Precision của một bộ phân loại mèo là tỉ lệ chính xác trong số ảnh được phân loại (là mèo) trong tập phát triển (hoặc tập kiểm tra). Trong khi đó Recall là tỉ lệ của số ảnh mèo ở trong tập phát triển (hoặc tập kiểm tra) được phân loại chính xác (là mèo). Thường có một sự đánh đổi giữa việc có chỉ số precision cao và chỉ số recall cao.

[4] Nếu bạn muốn đọc thêm về chỉ số F1, xem https://en.wikipedia.org/wiki/F1_score. Chỉ số F1 là "trung bình điều hoà" của Precision và Recall, được tính bằng 2/((1/Precision) + (1/Recall))

9. Phép đo để tối ưu và phép đo thỏa mãn

Đây là một cách khác để kết hợp nhiều phép đánh giá.

Giả sử bạn quan tâm đến cả độ chính xác lẫn thời gian chạy của một thuật toán học. Bạn cần phải chọn trong ba bộ phân loại sau:

Bộ phân loại	Độ chính xác	Thời gian chạy
A	90%	80ms
B	92%	95ms
C	95%	1,500ms

Việc tạo ra một phép đo đơn trị bằng cách gộp độ chính xác và thời gian chạy vào trong cùng một công thức khá là gượng ép, chẳng hạn:

Độ chính xác - 0.5*(Thời gian chạy)

Thay vào đó, bạn có thể: Trước hết, hãy định nghĩa thời gian chạy như thế nào là "chấp nhận được" - ví dụ mức dưới 100ms. Sau đó hãy cực đại hóa độ chính xác, với ràng buộc là bộ phân loại vẫn đảm bảo yêu cầu về thời gian chạy. Ở đây, thời gian chạy là một "phép đo thỏa mãn" —- bộ phân loại chỉ cần "đủ tốt" trên thang đo này, tức chỉ tốn tối đa 100ms để chạy xong. Độ chính xác mới là "phép đo để tối ưu".

Nếu bạn phải cân bằng giữa N tiêu chí khác nhau, ví dụ như kích thước file nhị phân của mô hình (điều này quan trọng với các ứng dụng di động vì người dùng không muốn tải các ứng dụng có kích thước lớn), thời gian chạy, và độ chính xác, bạn có thể cân nhắc đặt N-1 trong số đó làm các phép đo cần "thỏa mãn". Tức là bạn chỉ cần yêu cầu chúng đạt tới một giá trị xác định. Sau đó coi tiêu chí còn lại là phép đo để "tối ưu". Ví dụ như đặt mức ngưỡng chấp nhận được cho kích thước file nhị phân và thời gian chạy, sau đó tối ưu độ chính xác mà vẫn thỏa mãn các điều kiện các ràng buộc trên.

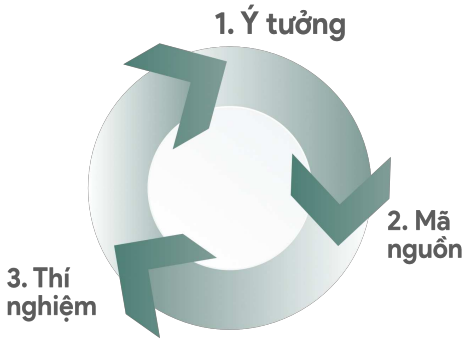
Ví dụ cuối cùng, giả sử bạn cần xây dựng một thiết bị phần cứng có microphone để nghe người dùng nói một từ "đánh thức" đặc biệt nào đó để đánh thức hệ thống. Ví dụ về từ đánh thức có: Amazon Echo với "Alexa"; Apple Siri với "Hey Siri"; Android với "Hey Google" hay các ứng dụng của Baidu với "Hello Baidu". Bạn quan tâm đến cả tần suất dương tính giả (hay báo động nhầm) -- tần suất mà hệ thống thức dậy khi không ai nói cụm đánh thức -- cũng như tần suất âm tính giả (hay bỏ sót) -- tần suất hệ thống không thức dậy khi có người nói cụm đánh thức. Một mục tiêu khả dĩ cho hệ thống này là tối thiểu hóa tần suất âm tính giả (phép đo để tối ưu), với điều kiện chỉ có tối đa một báo động nhầm cho mỗi 24 giờ hoạt động (phép đo thỏa mãn).

Một khi nhóm của bạn đã thống nhất phép đánh giá nào cần được tối ưu, cả nhóm sẽ đạt tiến độ nhanh hơn.

10. Xây dựng một tập phát triển và một phép đo sẽ tăng tốc quá trình làm việc

Thật sự rất khó để biết trước phương án tiếp cận nào là tốt nhất cho một vấn đề mới. Kể cả những nhà nghiên cứu học máy dày dặn kinh nghiệm cũng thường thử nghiệm cả chục ý tưởng mới khám phá ra một hướng đi thoả đáng. Khi xây dựng một hệ thống học máy, tôi thường:

1. Bắt đầu bằng một vài *ý tưởng* về cách xây dựng hệ thống đó.
2. Hiện thực hóa ý tưởng dưới dạng mã nguồn.
3. Tiến hành một **thí nghiệm** để đo mức hiệu quả của ý tưởng. (Thường thì những ý tưởng đầu tiên của tôi sẽ không hoạt động!) Học được từ những kết quả đó, tôi quay lại thử nghiệm thêm những ý tưởng mới, và cứ thế lặp lại cả quy trình.



Đây là một quy trình lặp đi lặp lại. Bạn thực hiện vòng lặp này càng nhanh thì tốc độ cải tiến kết quả càng cao. Đó là lý do tại sao có tập phát triển/thử nghiệm và một phép đo là rất quan trọng: Việc đánh giá chất lượng của mỗi ý tưởng trên tập phát triển giúp ta xác định được liệu mình có đang đi đúng hướng.

Ngược lại, giả sử bạn không có một tập phát triển và phép đo nào cụ thể. Như vậy mỗi khi nhóm của bạn phát triển một bộ phân loại mèo mới, bạn sẽ lại phải tích hợp nó vào ứng dụng, và thử nghiệm ứng dụng đó một vài tiếng để kiểm tra liệu bộ phân loại mới có tốt hơn hay không. Như vậy sẽ cực kì chậm! Đồng thời, nhóm của bạn sẽ rất khó nhận ra sự khác biệt nếu độ chính xác chỉ cải thiện từ 95.0% lên 95.1%, bạn sẽ không thể nhận ra những cải tiến 0.1% đó chỉ qua việc dùng thử ứng dụng. Thế nhưng, phần lớn những cải tiến đến từ việc tích lũy nhiều bước cải thiện 0.1% này. Việc có một tập phát triển và phép đo cho phép bạn nhanh chóng phát hiện ra ý tưởng nào đang đem lại những cải tiến nhỏ (hoặc lớn) và giúp bạn ra quyết định ý tưởng nào cần cải thiện thêm hoặc loại bỏ.

11. Khi nào cần thay đổi tập phát triển/kiểm tra và các phép đo

Khi bắt đầu một dự án, tôi luôn cố gắng chọn tập phát triển/kiểm tra thật nhanh để tạo một mục tiêu rõ ràng cho cả nhóm.

Tôi thường yêu cầu các nhóm của tôi xác định tập phát triển/kiểm tra và một phép đo ban đầu hiếm khi lâu hơn một tuần. Tốt hơn hết là có được tập phát triển/kiểm tra và phép đánh giá đơn trị, dù là chưa hoàn hảo để bắt đầu nhanh chóng, hơn là suy nghĩ quá nhiều về chúng. Tuy nhiên, thời hạn một tuần không áp dụng với các ứng dụng đã phát triển. Ví dụ, chống thư rác là một ứng dụng học sâu đã phát triển. Tôi từng thấy các nhóm làm việc với những hệ thống đã phát triển dành hàng tháng để tạo được những tập phát triển/kiểm tra tốt hơn.

Nếu sau đó bạn nhận ra rằng tập phát triển/kiểm tra hoặc phép đo ban đầu không phù hợp với mục tiêu đặt ra, bằng mọi giá hãy thay đổi chúng một cách nhanh chóng. Chẳng hạn, nếu phép đo trên tập phát triển xếp hạng bộ phân loại A tốt hơn bộ phân loại B, nhưng nhóm nghĩ rằng bộ phân loại B thực ra lại tốt hơn cho sản phẩm của bạn, đây có thể là dấu hiệu cần thay đổi tập phát triển/kiểm tra hoặc phép đánh giá.

Ba nguyên nhân chính khiến tập phát triển/phép đánh giá xếp hạng bộ phân loại A cao hơn:

1. Phân phối thực tế mà bạn cần làm tốt khác với phân phối của tập phát triển/kiểm tra.



Giả sử tập phát triển/kiểm tra ban đầu chứa chủ yếu ảnh mèo trưởng thành. Sau khi

ra mắt ứng dụng, bạn nhận ra rằng người dùng lại tải lên ảnh mèo con nhiều hơn dự tính. Khi đó, phân phối của tập phát triển/kiểm tra không đại diện cho phân phối thực tế mà cần bạn hướng tới. Trong trường hợp này, bạn cần cập nhật tập phát triển/kiểm tra sao cho chúng có tính đại diện hơn.

2. Mô hình của bạn đã quá khớp tập phát triển.

Quá trình đánh giá ý tưởng trên tập phát triển được lặp đi lặp lại khiến thuật toán dần "quá khớp" tập dữ liệu này. Sau khi phát triển xong, bạn sẽ đánh giá mô hình trên tập kiểm tra. Nếu bạn thấy rằng chất lượng trên tập phát triển tốt hơn nhiều so với chất lượng trên tập kiểm tra, đây là dấu hiệu bạn đã quá khớp tập phát triển. Trong trường hợp này, bạn hãy tạo một tập phát triển mới hoàn toàn.

Nếu bạn cần theo dõi tiến độ của nhóm, bạn cũng có thể đánh giá hệ thống trên tập kiểm tra thường xuyên, chẳng hạn mỗi tuần hoặc mỗi tháng một lần. Tuy nhiên, không được sử dụng tập kiểm tra để đưa ra bất kì quyết định nào liên quan tới thuật toán, bao gồm việc quay lui về hệ thống trước đó. Nếu làm vậy, bạn sẽ bắt đầu quá khớp tập kiểm tra và không thể tiếp tục dựa vào nó để tạo ra một đánh giá hoàn toàn không thiên lệch cho chất lượng của hệ thống (bạn sẽ cần đánh giá như vậy khi bạn xuất bản công trình nghiên cứu hoặc là để đưa ra những quyết định kinh doanh quan trọng dựa trên phép đo này).

3. Phép đo không phù hợp với mục tiêu tối ưu của dự án.

Giả sử trong ứng dụng mèo, phép đo của bạn là độ chính xác phân loại. Phép đo này hiện tại xếp hạng bộ phân loại A tốt hơn bộ phân loại B. Tuy nhiên, giả sử bạn thử cả hai thuật toán, và nhận ra rằng bộ phân loại A thi thoảng chấp nhận những bức ảnh khiêu dâm. Ngay cả khi bộ phân loại A chính xác hơn, ấn tượng xấu gây ra bởi một vài bức ảnh khiêu dâm đồng nghĩa với việc chất lượng của nó là không chấp nhận được. Bạn sẽ làm gì?

Ở đây, phép đo thất bại trong việc xác định được thực tế Thuật toán B tốt hơn Thuật toán A cho sản phẩm của bạn. Bởi vậy, bạn không thể dựa vào phép đo này để chọn thuật toán tốt nhất. Đây là lúc phải thay đổi phép đo. Ví dụ, bạn có thể thay đổi phép đo sao cho nó "phạt" thật nặng nếu một thuật toán chấp nhận ảnh khiêu dâm. Tôi khuyên bạn chọn một phép đo mới và sử dụng nó để định nghĩa lại thật rõ ràng mục tiêu của nhóm, hơn là cứ tiếp tục chọn thủ công trong số các bộ phân loại khi không có một phép đo đáng tin cậy.

Việc thay đổi tập phát triển/kiểm tra hoặc phép đo giữa dự án khá là phổ biến. Có một tập phát triển/kiểm tra và phép đo ban đầu giúp bạn hoàn thành chu kỳ phát triển một cách nhanh chóng. Nếu bạn nhận ra rằng tập phát triển/kiểm tra hoặc phép đo không còn giúp nhóm đi đúng hướng, không sao cả! Chỉ cần thay chúng và đảm bảo cả nhóm đều biết về hướng đi mới này.

12. Điều cần nhớ: Thiết lập các tập phát triển và kiểm tra

- Chọn tập phát triển và kiểm tra từ một phân phối phản ánh được dữ liệu bạn dự tính sẽ cần xử lý trong tương lai và muốn hoạt động tốt với chúng. Phân phối này có thể không giống phân phối trên dữ liệu huấn luyện của bạn.
- Nếu có thể, hãy chọn tập phát triển và kiểm tra từ cùng một phân phối xác suất.
- Hãy chọn một phép đo đơn trị để tối ưu hóa. Nếu có nhiều mục tiêu cần quan tâm đến, hãy kết hợp chúng thành một công thức duy nhất (chẳng hạn như lấy trung bình các phép đo sai số) hoặc xác định ra phép đo thỏa mãn và phép đo để tối ưu.
- Học máy là một quá trình lặp đi lặp lại: Bạn có thể phải thử hàng tá ý tưởng trước khi tìm thấy một ý tưởng mà bạn hài lòng.
- Có tập phát triển/kiểm tra và một phép đo đơn trị giúp bạn nhanh chóng đánh giá các thuật toán và do đó lặp lại nhanh hơn.
- Khi bắt đầu trên một ứng dụng hoàn toàn mới, hãy nhanh chóng thiết lập tập phát triển/kiểm tra và một phép đo trong vòng một tuần. Với các ứng dụng đã phát triển, quá trình này có thể kéo dài hơn.
- Cách chia dữ liệu huấn luyện/kiểm tra với tỉ lệ 70%/30% theo kinh nghiệm cũ không áp dụng cho các bài toán với nhiều dữ liệu; tập phát triển và kiểm tra có thể chiếm ít hơn con số 30% rất nhiều.
- Tập phát triển của bạn phải đủ lớn để phát hiện các thay đổi có ý nghĩa đối với độ chính xác của thuật toán, nhưng không nhất thiết phải lớn hơn nhiều. Tập kiểm tra phải đủ lớn để cung cấp cho bạn ước lượng đáng tin cậy về chất lượng cuối cùng của hệ thống.
- Nếu tập phát triển và phép đo không còn chỉ cho nhóm của bạn đi đúng hướng, hãy nhanh chóng thay đổi chúng: (i) Nếu thuật toán đã quá khớp tập phát triển, hãy thu thập thêm dữ liệu cho tập này. (ii) Nếu phân phối xác suất thực tế mà bạn quan tâm khác với phân phối xác suất của tập phát triển/kiểm tra, hãy tạo tập phát triển và kiểm tra mới. (iii) Nếu phép đo không còn đo lường được điều quan trọng nhất với bạn, hãy thay đổi phép đo.

Phần 2: Phân tích lỗi cơ bản

13. Bạn mong muốn xây dựng một hệ thống phòng chống email rác mới. Nhóm của bạn có rất nhiều ý tưởng:

Thu thập một tập huấn luyện lớn về email rác. Ví dụ như thiết lập một Honeypot (Mồi nhử): cố ý gửi các địa chỉ email giả đến những spammer đã biết, và bạn có thể thu thập các tin nhắn rác mà họ gửi đến địa chỉ đó một cách tự động.

Phát triển những tính năng để hiểu được nội dung văn bản trong email.

Phát triển những tính năng để hiểu được các đặc tính của phông thư/nhãn thư từ email nhằm hiển thị tập hợp các máy chủ internet mà thư đã đi qua.

- và nhiều hơn thế.

Mặc dù tôi đã kinh qua rất nhiều trong việc phòng chống email rác, tôi vẫn sẽ gặp khó khăn khi phải chọn một trong các hướng đi trên. Điều này sẽ còn khó hơn nếu bạn không phải là một chuyên gia trong lĩnh vực này.

Vì vậy, bạn không nên bắt đầu bằng việc thiết kế và xây dựng một hệ thống hoàn hảo. Thay vào đó, hãy xây dựng và huấn luyện nhanh một hệ thống cơ bản -- có thể là trong vài ngày[5]. Ngay cả khi hệ thống cơ bản khác xa với hệ thống tốt nhất mà bạn có thể xây dựng, khám phá cách thức hoạt động của hệ thống cơ bản này vẫn đem lại nhiều giá trị: bạn sẽ nhanh chóng tìm ra được những dấu hiệu cho những hướng đi hứa hẹn nhất để đầu tư thời gian. Trong những chương tiếp theo sẽ chỉ cho bạn cách tìm thấy những dấu hiệu này.



CHÚ THÍCH:

Lời khuyên này dành cho những độc giả có mong muốn xây dựng các ứng dụng AI, hơn là những người có mục tiêu là xuất bản những bài báo học thuật. Tôi sẽ quay trở lại với chủ đề nghiên cứu này sau.

14. Phân tích lỗi: đánh giá ý tưởng dựa trên tập phát triển



Khi kiểm thử ứng dụng nhận dạng mèo, bạn thấy rằng một số bức ảnh chó bị nhận nhầm. Nhìn chúng tương đối giống mèo!

Một thành viên trong nhóm đề xuất tích hợp phần mềm của bên thứ ba nhằm giúp hệ thống phân biệt tốt hơn các bức ảnh chó. Có thể mất một tháng để hoàn thành quá trình tích hợp và người đề xuất rất hào hứng. Liệu bạn có nên yêu cầu thành viên đó bắt đầu công việc?

Trước khi đầu tư cả một tháng, bạn nên ước lượng việc này có thể cải thiện độ chính xác của hệ thống tới mức nào. Từ đó, bạn sẽ có thể quyết định xem có đáng bỏ ra chừng đó thời gian vào việc phát triển hay là dành nó cho những việc khác.

Cụ thể, bạn có thể làm theo các bước sau:

1. Thu thập 100 mẫu trong tập phát triển mà ứng dụng của bạn phân loại nhầm — không phải mèo nhưng được phân loại là mèo và ngược lại.
2. Nhìn vào những mẫu trên và đếm xem bao nhiêu trong số đó là ảnh chó.

Quá trình nhìn vào những mẫu bị phân loại nhầm được gọi là **phân tích lỗi**. Trong ví dụ này, nếu bạn nhận thấy rằng chỉ 5% lỗi là chó nhầm thành mèo thì cho dù cải thiện thuật toán theo hướng tích hợp phần mềm nhận dạng chó vào ứng dụng, bạn không thể loại bỏ quá 5% số ảnh bị nhận dạng sai. Nói cách khác, 5% là "cận trên" (số lượng tối đa có thể đạt được) cho mức độ cải thiện mà hướng đi trên có thể giúp cho hệ thống. Nếu như độ chính xác ban đầu của ứng dụng là 90% (10% lỗi), việc cải thiện chỉ làm cho hệ thống của bạn đạt được độ chính xác mới là 90,5% (9,5% lỗi, ít hơn 5% so với số lượng lỗi ban đầu).

Ngược lại, nếu bạn nhận thấy rằng 50% lỗi là do chó bị nhầm thành mèo thì bạn có thể tự tin rằng phương án được đề xuất sẽ có tác động lớn. Nó có thể cải thiện đáng kể độ chính xác của hệ thống từ 90% lên 95% (giảm 50% tổng số lỗi, từ 10% xuống 5%).

Cách phân tích lỗi đơn giản ở trên giúp bạn ước lượng nhanh hiệu quả của việc tích hợp phần mềm nhận dạng chó của bên thứ ba vào hệ thống nhận dạng mèo. Đây cũng là cơ sở định lượng để bạn lựa chọn xem có nên đi theo hướng này hay không.

Việc **phân tích lỗi** thường giúp bạn nhìn thấy được triển vọng của những hướng giải

quyết khác nhau. Tôi thấy nhiều kỹ sư tiến hành phân tích lỗi môt cách miễn cưỡng. Dường như đối với họ ngay lập tức thực hiện một số ý tưởng sẽ thú vị hơn là tự hỏi xem ý tưởng đó có thật sự đáng để bạn bỏ thời gian thực hiện. Đây là một lỗi phổ biến: nó có thể gây lãng phí hàng tháng chỉ để nhận ra rằng sự cải thiện là không đáng kể.

Quan sát 100 mẫu để phân tích lỗi không tốn nhiều thời gian. Kể cả khi bạn bỏ ra một phút để kiểm tra từng ảnh, thời gian tổng cộng vẫn nhỏ hơn hai giờ. Nếu như ý tưởng kia không tốt, bỏ ra hai giờ phân tích lỗi này có thể giúp bạn tiết kiệm được một tháng.

Việc phân tích lỗi là quá trình kiểm tra các mẫu trong tập phát triển bị phân loại nhầm, từ đó bạn có thể hiểu được nguyên nhân. Hiểu rõ nguyên nhân tạo ra lỗi sẽ giúp bạn nhìn ra những hướng giải quyết mới mà chúng ta sẽ thảo luận ở phần sau. Một số chương tiếp theo sẽ trình bày những "best practices" được dùng để phân tích lỗi.

15. Đánh giá song song các ý tưởng trong quá trình phân tích lỗi

Nhóm của bạn có một số ý tưởng cải thiện bộ phát hiện mèo:

- Sửa lỗi nhận dạng chó thành mèo trong thuật toán.
- Sửa lỗi nhận dạng thú họ mèo (sư tử, báo, v.v) thành mèo nhà (thú nuôi).
- Cải thiện chất lượng của hệ thống trên ảnh mờ.
- ...

Bạn có thể đánh giá song song tất cả các ý kiến trên một cách hiệu quả. Tôi thường tạo một bảng và điền vào đó khi phân tích ~100 ảnh phân loại nhầm trong tập phát triển. Tôi cũng ghi chú ngắn gọn để ghi nhớ những trường hợp đặc biệt. Để minh họa cho quá trình này, bạn có thể tham khảo bảng được tạo ra từ một tập phát triển nhỏ với bốn mẫu dưới đây:

Ảnh	Chó	Thú Họ Mèo	Ảnh Mờ	Ghi chú
1	✔			Chó pitbull có màu lạ
2			✔	
3		✔	✔	Ảnh sư tử chụp ở sở thú trong một ngày mưa
4		✔		Một con báo bị khuất sau cây
Tổng %	25%	50%	50%	

Ảnh #3 ở trên có cả hai cột Thú Họ Mèo và Ảnh Mờ được đánh dấu. Thêm vào đó, bởi vì một mẫu có thể nằm ở nhiều hạng mục, tổng phần trăm của hàng cuối có thể không đạt 100%.

Mặc dù bạn có thể tạo từ trước các hạng mục (Chó, Thú Họ Mèo, Ảnh Mờ) và sau đó phân loại các mẫu thủ công, trong quá trình phân tích mẫu, bạn có thể nảy ra những ý tưởng về các hạng mục mới. Ví dụ: bạn phân loại hàng chục bức ảnh và nhận ra nhiều lỗi xảy ra ở những tấm ảnh chỉnh bởi bộ lọc Instagram. Bạn có thể quay lại và thêm cột "Instagram" vào bảng. Bằng cách nhìn vào từng mẫu mà thuật toán phân loại nhầm và đặt câu hỏi làm thế nào/liệu rằng con người có thể nhận dạng mẫu này một cách chính xác, nhiều khả năng là bạn sẽ tìm được các hạng mục lỗi và giải pháp mới.

Những hạng mục lỗi hữu ích nhất sẽ là những lỗi mà bạn có thể khắc phục. Ví dụ, hạng mục Instagram sẽ là hữu ích nhất để thêm vào nếu bạn biết cách "đảo ngược" bộ lọc Instagram và phục hồi ảnh gốc. Tuy nhiên bạn không nhất thiết phải giới hạn bản thân chỉ với những hạng mục mà bạn biết cách cải thiện; mục tiêu của quá trình này là xây dựng một góc nhìn rõ hơn về những đặc trưng tiềm năng mà bạn nên tập

trung vào.

Phân tích lỗi là một quá trình lặp đi lặp lại. Đừng lo nếu bạn bắt đầu mà vẫn chưa nghĩ được hạng mục nào. Bạn sẽ có thêm ý tưởng về các hạng mục lỗi mới sau khi phân tích một vài tấm ảnh. Sau khi phân loại thủ công một số hình ảnh, bạn có thể nghĩ ra các hạng mục mới và đối chiếu lại các mẫu ảnh theo hạng mục mới đó.

Giả sử bạn hoàn thành việc phân tích lỗi 100 mẫu bị phân loại nhầm trên tập phát triển và có được kết quả như sau:

Ảnh	Chó	Thú Họ Mèo	Ảnh Mờ	Ghi chú
1	✔			Chó pitbull có màu lạ
2			✔	
3		✔	✔	Ảnh sư tử chụp ở sở thú trong một ngày mưa
4		✔		Một con báo bị khuất sau cây
...	...	...	...	...
Tổng %	8%	43%	61%	

Bạn thấy rằng việc khắc phục lỗi phân loại nhầm trên hạng mục Chó có thể loại bỏ tối đa 8% lỗi. Khắc phục các lỗi trên hạng mục Thú Họ Mèo và Ảnh Mờ có thể loại bỏ được nhiều lỗi hơn. Vì vậy bạn có thể chọn một trong hai hạng mục trên để tập trung vào. Nếu nhóm của bạn có đủ nhân lực để khắc phục nhiều hạng mục lỗi song song, bạn có thể phân công một số kỹ sư khắc phục lỗi trên hạng mục Thú Họ Mèo, những người còn lại khắc phục lỗi trên hạng mục Ảnh Mờ.

Phân tích lỗi không tạo ra một công thức toán học cứng nhắc cho bạn biết hạng mục nào có độ ưu tiên cao nhất. Bạn cũng cần đánh giá khả năng cải thiện có thể đạt được trên các hạng mục cũng như khối lượng công việc cần thiết để giải quyết từng hạng mục đó.

16. Dọn dẹp những mẫu bị gán nhãn nhầm trong tập phát triển và tập kiểm tra

Trong quá trình phân tích lỗi, bạn có thể nhận thấy rằng một vài mẫu trong trong tập phát triển bị gán nhãn nhầm. Khi nói "bị gán nhãn nhầm", ý tôi là những tấm ảnh đã bị gán nhãn nhầm bởi người dán nhãn trước cả khi chạy thuật toán. Hay nói cách khác, nhãn lớp của một mẫu (x,y) có giá trị y sai. Ví dụ, có thể một số ảnh không chứa mèo bị gán nhãn nhầm thành có mèo và ngược lại. Nếu bạn nghi ngờ rằng tỷ lệ những ảnh bị gán nhãn nhầm là đáng kể, hãy thêm một hạng mục để theo dõi tỷ lệ các mẫu bị gán nhãn nhầm:

Ảnh	**Chó**	**Thú họ mèo**	**Ảnh mờ**	**Dán nhãn sai**	**Ghi chú**
...					
98				✔	Người dán nhãn đã bỏ qua con mèo ở phần nền
99		✔			
100				✔	Bức vẽ của con mèo, không phải con mèo thật
Tổng %	8%	43%	61%	6%	

Vậy bạn có nên sửa lại những nhãn sai trong tập phát triển không? Hãy nhớ rằng mục tiêu của tập phát triển là giúp bạn nhanh chóng đánh giá các thuật toán nhờ đó bạn có thể biết liệu Thuật toán A hay Thuật toán B là tốt hơn. Nếu tỷ lệ bị gán nhãn nhầm trong tập phát triển cản trở khả năng ra những quyết định này của bạn, thì sẽ đáng để bỏ thời gian ra để sửa lại những nhãn bị gán nhầm của tập phát triển.

Để ví dụ, giả sử chất lượng bộ phân loại của bạn là:

- Độ chính xác tổng thể trên tập phát triển..... 90% (10% lỗi tổng thể.)
- Những lỗi gây ra bởi các mẫu bị gán nhãn nhầm..... 0,6% (6% các lỗi trong tập phát triển.)
- Những lỗi do các nguyên nhân khác..... 9,4% (94% các lỗi trong tập phát triển)

Ở đây, tỷ lệ 0,6% sai do gán nhầm nhãn có thể không quá đáng kể so với tỷ lệ 9,4% các lỗi mà bạn có thể cải thiện. Không có một tác hại nào trong việc sửa thủ công những ảnh bị gán nhẫn nhầm trong tập phát triển cả, nhưng nó không quá quan trọng để làm vậy: Việc bạn không biết liệu hệ thống của mình có 10% hay 9,4% lỗi chung là có thể chấp nhận được.

Giả sử bạn tiếp tục cải thiện bộ nhận dạng mèo và đạt chất lượng:

- Độ chính xác tổng thể trên tập phát triển..... 98,0% (2,0% lỗi tổng thể.)

- Những lỗi gây ra do các mẫu bị gán nhãn nhầm..... 0,6%. (30% các lỗi trong tập phát triển.)
- Những lỗi do các nguyên nhân khác..... 1,4% (70% các lỗi trong tập phát triển)

30% lỗi của bạn đến từ những ảnh bị gán nhãn nhầm trong tập phát triển, việc này thêm một lượng đáng kể lỗi vào các đánh giá độ chính xác của bạn. Trong trường hợp này, cải thiện chất lượng của các nhãn trong tập phát triển là một việc đáng làm. Xử lý những mẫu bị gán nhãn nhầm sẽ giúp bạn biết được lỗi của bộ phân loại gần với 1,4% hay 2% -- một sự khác biệt đáng kể.

Thông thường, trong giai đoạn đầu, bỏ qua một vài nhãn sai trong tập phát triển/kiểm tra là chấp nhận được. Khi hệ thống được cải thiện, số mẫu bị gán nhãn sai dần chiếm tỷ lệ tương đối lớn trong tập lỗi, lúc này ta mới tiến hành sửa lại các nhãn đó.

Chương trước đã hướng dẫn cách bạn có thể cải thiện các hạng mục lỗi như Chó, Thú Họ Mèo và Ảnh Mờ qua những cải tiến về thuật toán. Bạn đã học trong chương này rằng bạn cũng có thể xử lý thêm hạng mục Bị Gán Nhãn Sai nữa -- thông qua cải thiện các nhãn của dữ liệu.

Bất kể quy trình bạn áp dụng để sửa các nhãn trong tập huấn luyện là gì, hãy nhớ áp dụng cùng một quy trình cho các nhãn của tập kiểm tra, để đảm bảo tập phát triển và kiểm tra vẫn được lấy ra từ cùng một phân phối. Chỉnh sửa các tập phát triển và kiểm tra cùng nhau sẽ giúp tránh được những vấn đề chúng ta đã bàn trong Chương 6, khi nhóm của bạn tối ưu chất lượng cho tập phát triển để rồi phát hiện ra sau đó là chúng đang được đánh giá dựa trên một tiêu chuẩn khác dựa trên một tập kiểm tra khác.

Nếu bạn quyết định cải thiện chất lượng nhãn, hãy xem xét việc kiểm tra kỹ các nhãn của những mẫu mà hệ thống của bạn đã phân loại nhầm cũng như các nhãn của những mẫu mà nó đã phân loại chính xác. Rất có thể là cả nhãn gốc và thuật toán học máy của bạn đều đã sai trên một mẫu. Nếu bạn chỉ sửa những nhãn của mẫu mà hệ thống đã phân loại nhầm, bạn có thể đã gây ra thiên lệch trong đánh giá. Nếu bạn có 1.000 mẫu trong tập phát triển, và nếu bộ phân loại của bạn có 98,0% độ chính xác, sẽ dễ hơn khi kiểm tra 20 mẫu đã bị phân loại nhầm hơn là cả 980 mẫu được phân loại chính xác. Bởi vì trên thực tế, sẽ dễ hơn khi chỉ kiểm tra những mẫu bị phân loại nhầm, do đó sự thiên lệch sẽ lẻn vào một vài tập phát triển. Sự thiên lệch này là chấp nhận được nếu bạn chỉ quan tâm vào việc phát triển một sản phẩm hay một ứng dụng, nhưng nó sẽ là một vấn đề nếu bạn định sử dụng kết quả trong một bài báo nghiên cứu khoa học hay cần một phép đo hoàn toàn không thiên lệch cho độ chính xác của tập kiểm tra.

17. Nếu bạn có một tập phát triển lớn, chia nó thành hai tập con và chỉ phân tích trên một tập

Giả sử bạn có một tập phát triển lớn gồm 5000 mẫu, với tỉ lệ lỗi là 20%. Ở đây, thuật toán của bạn đang phân loại nhầm khoảng 1000 mẫu ảnh của tập phát triển. Sẽ rất lâu để phân tích thủ công 1000 ảnh này, vì vậy, chúng ta có thể quyết định không sử dụng tất cả 1000 ảnh đó trong phân tích lỗi.

Trong trường hợp này, tôi sẽ chia tập phát triển thành hai tập con riêng biệt: một tập sẽ được phân tích thủ công (bằng cách nhìn vào từng mẫu), tập còn lại thì không. Thuật toán sẽ quá khớp phần được phân tích thủ công nhanh hơn. Phần còn lại có thể được sử dụng để điều chỉnh tham số.



Hãy cùng tiếp tục với ví dụ ở trên: ví dụ thuật toán đang phân loại nhầm 1000 mẫu trên tổng số 5000 mẫu trong tập phát triển. Giả sử chúng ta muốn kiểm tra một cách thủ công 100 mẫu bị phân loại nhầm để phân tích lỗi (10% tổng số lỗi). Bạn nên chọn ra 10% mẫu trong tập phát triển một cách ngẫu nhiên và đặt nó vào trong một tập mà chúng ta sẽ gọi là **tập phát triển Eyeball** để tự nhắc chúng ta rằng chúng ta sẽ trực tiếp nhìn vào bằng mắt. (Đối với những dự án nhận diện giọng nói mà bạn phải nghe các clip audio, có lẽ bạn sẽ gọi tập này là tập phát triển Ear). Tập phát triển Eyeball chứa 500 mẫu, trong đó chúng ta kỳ vọng thuật toán sẽ phân loại nhầm khoảng 100 mẫu.

Tập con thứ hai của tập phát triển, được gọi là **tập phát triển Blackbox**, sẽ chứa 4500 mẫu còn lại. Bạn có thể sử dụng tập phát triển Blackbox để đánh giá các bộ phân loại một cách tự động bằng cách đo tỉ lệ lỗi của chúng. Bạn cũng có thể sử dụng tập này để lựa chọn giữa các thuật toán hoặc điều chỉnh các siêu tham số. Tuy nhiên, bạn nên tránh trực tiếp phân tích thủ công trên tập này. Chúng ta sử dụng thuật ngữ "Blackbox" vì chúng ta chỉ sử dụng tập con này để thu về những đánh giá "Blackbox" của các bộ phân loại.



Tại sao chúng ta lại chia tập phát triển một cách riêng biệt thành tập phát triển Eyeball và tập phát triển Blackbox? Bởi vì bạn sẽ hiểu rõ hơn về các mẫu trong tập phát triển Eyeball, bạn sẽ bắt đầu quá khớp tập phát triển đó nhanh hơn. Nếu bạn thấy chất lượng của mô hình trên tập phát triển Eyeball đang tăng nhanh hơn nhiều so với trên tập phát triển Blackbox, bạn đã quá khớp tập phát triển Eyeball. Trong trường hợp này, bạn có thể phải loại bỏ tập Eyeball đi, tìm một tập Eyeball khác thay thế bằng cách chuyển các mẫu từ tập phát triển Blackbox thành tập phát triển Eyeball mới, hoặc thu thập những mẫu có nhãn mới.

Việc phân chia tập phát triển thành hai tập riêng biệt -- tập phát triển Eyeball và tập phát triển Blackbox -- cho bạn biết khi nào việc phân tích lỗi thủ công đang khiến bạn quá khớp tập Eyeball.

18. Tập phát triển Eyeball và Blackbox nên lớn như thế nào?



Tập phát triển Eyeball phải đủ lớn để giúp bạn có cái nhìn về các hạng mục lỗi chính của thuật toán. Nếu bạn đang làm một tác vụ mà con người làm tốt (chẳng hạn như nhận diện mèo trong các ảnh), dưới đây là một vài hướng dẫn sơ bộ.

- Một tập phát triển Eyeball ở đó các bộ phân loại tạo ra 10 lỗi có thể được coi là rất nhỏ. Với chỉ 10 lỗi, rất khó để ước lượng chuẩn xác ảnh hưởng của những hạng mục lỗi khác nhau. Nhưng nếu bạn có rất ít dữ liệu và không thể tăng số mẫu trong tập phát triển Eyeball, việc này vẫn tốt hơn so với không có gì và sẽ giúp ích đối với việc sắp xếp ưu tiên của dự án.
- Nếu bộ phân loại tạo ra khoảng 20 lỗi trong tập phát triển Eyeball, bạn sẽ bắt đầu cảm nhận được sơ bộ về các nguồn lỗi chính.
- Với khoảng 50 lỗi, bạn sẽ có cảm nhận tốt về các nguồn lỗi chính.
- Với khoảng 100 lỗi, bạn sẽ cảm nhận được rất tốt các nguồn lỗi chính. Tôi đã chứng kiến nhiều người phân tích thủ công nhiều lỗi hơn, đôi khi tới 500 lỗi. Điều đó không gây hại miễn là bạn có đủ dữ liệu.

Giả sử bộ phân loại của bạn có tỷ lệ lỗi 5%. Để đảm bảo bạn có khoảng 100 mẫu bị phân loại sai trong tập phát triển Eyeball, tập phát triển Eyeball sẽ phải có khoảng 2.000 mẫu (bởi vì 0,05 * 2.000 = 100). Tỷ lệ lỗi do bộ phân loại gây ra càng thấp, tập phát triển Eyeball càng phải lớn để có được một tập lỗi đủ lớn cho phân tích.

Nếu bạn đang làm việc trong một tác vụ mà ngay cả con người cũng không thể làm tốt, thì việc kiểm tra tập phát triển Eyeball sẽ không hữu ích như trong trường hợp trên vì khó hình dung tại sao thuật toán không phân loại mẫu một cách chính xác. Trong trường hợp này, bạn có thể bỏ qua việc thiết lập tập phát triển Eyeball. Chúng ta thảo luận hướng dẫn cho những vấn đề này trong một chương sau.



Tập phát triển Blackbox thì sao? Trước đây chúng ta đã khẳng định rằng các tập phát triển khoảng 1.000-10.000 mẫu là khá phổ biến. Để củng cố nhận định đó, một tập phát triển Blackbox gồm 1.000-10.000 mẫu thường cung cấp đủ dữ liệu để tinh chỉnh siêu tham số và lựa chọn giữa các mô hình, mặc dù có ít bất lợi khi có nhiều dữ liệu hơn. Một tập phát triển Blackbox với 100 mẫu sẽ nhỏ nhưng vẫn hữu ích.

Nếu bạn có một tập phát triển nhỏ thì bạn có thể không đủ dữ liệu để phân chia thành tập phát triển Eyeball và tập phát triển Blackbox đủ lớn để đáp ứng mục đích sử dụng của chúng. Thay vào đó, toàn bộ tập phát triển của bạn có thể phải được sử dụng như là tập phát triển Eyeball, tức là bạn sẽ kiểm tra thủ công toàn bộ tập phát triển.

Giữa tập phát triển Eyeball và tập phát triển Blackbox, tôi cho rằng tập phát triển Eyeball quan trọng hơn (giả định bạn đang giải quyết một vấn đề mà con người có thể giải quyết tốt và việc kiểm tra mẫu giúp bạn hiểu rõ hơn). Nếu bạn chỉ có một tập phát triển Eyeball, bạn có thể thực hiện phân tích lỗi, lựa chọn mô hình và tinh chỉnh siêu tham số, tất cả trên tập dữ liệu này. Nhược điểm của việc chỉ có một tập phát triển Eyeball là nguy cơ quá khớp trên tập phát triển là lớn hơn.

Nếu bạn có quyền truy cập vào nhiều dữ liệu thì kích thước của tập phát triển Eyeball sẽ chủ yếu dựa trên bao nhiêu mẫu mà bạn có thời gian để phân tích thủ công. Ví dụ, tôi hiếm khi thấy ai phân tích thủ công hơn 1.000 lỗi.

19. Điều cần nhớ: Phân tích lỗi cơ bản

- Khi bạn bắt đầu một dự án mới, đặc biệt nếu bạn không phải là chuyên gia trong lĩnh vực đó, sẽ rất khó để đoán chính xác những hướng giải quyết triển vọng nhất.
- Vì vậy đừng cố bắt đầu với việc thiết kế và xây dựng một hệ thống hoàn hảo. Thay vào đó, hãy xây dựng và huấn luyện một hệ thống cơ bản một cách nhanh nhất có thể -- thậm chí chỉ trong một vài ngày. Sau đó, sử dụng phân tích lỗi để xác định những hướng đi triển vọng và từ đó lặp đi lặp lại việc cải thiện thuật toán của bạn.
- Thực hiện phân tích lỗi bằng cách kiểm tra thủ công khoảng 100 mẫu trong tập phát triển mà thuật toán phân loại sai và điểm qua những hạng mục lỗi chính. Sử dụng thông tin này để sắp xếp thứ tự ưu tiên các loại lỗi cần khắc phục.
- Xem xét việc tách tập phát triển thành một tập phát triển Eyeball cho việc kiểm tra thủ công, và một tập phát triển Blackblox mà bạn sẽ không kiểm tra thủ công. Nếu chất lượng trên tập phát triển Eyeball tốt hơn rất nhiều so với trên tập phát triển Blackbox, bạn đã quá khớp tập phát triển Eyeball và nên xem xét việc thu thập thêm dữ liệu cho tập này.
- Tập phát triển Eyeball nên đủ lớn để số lượng mẫu mà thuật toán của bạn phân loại sai đủ cho bạn phân tích. Một tập phát triển Blackbox khoảng 1.000-10.000 mẫu là đủ cho rất nhiều những ứng dụng.
- Nếu tập phát triển của bạn không đủ lớn để tách ra theo cách này, hãy lấy toàn bộ tập phát triển làm một tập phát triển Eyeball dành cho việc phân tích lỗi thủ công, chọn mô hình, và điều chỉnh siêu tham số.

Phần 3: Độ chệch và Phương sai

20. Độ chệch và Phương sai: Hai nguồn lớn của lỗi

Giả sử khi huấn luyện, tập phát triển và tập kiểm tra có cùng phân phối. Khi đó bạn cần luôn cố gắng thu thập thêm dữ liệu huấn luyện, vì dù sao điều đó cũng chỉ giúp cải thiện chất lượng, đúng không?

Mặc dù có thêm dữ liệu không gây ảnh hưởng xấu, thì thật không may, điều này không phải lúc nào cũng mang lại hiệu quả nhiều như mong đợi. Việc thu thập thêm dữ liệu có thể trở nên lãng phí thời gian. Vậy làm thế nào để quyết định khi nào nên hoặc không nên thêm dữ liệu?

Có hai nguồn lỗi chính trong học máy: độ chệch và phương sai. Hiểu được chúng sẽ giúp bạn quyết định liệu việc thêm dữ liệu hay áp dụng các kỹ thuật khác để cải thiện chất lượng của mô hình có đáng để bỏ thời gian ra không.

Giả sử bạn hy vọng xây dựng được một bộ nhận dạng mèo với 5% lỗi. Hiện tại, tập huấn luyện và tập phát triển có tỉ lệ lỗi lần lượt là 15% và 16%. Trong trường hợp này, việc thêm dữ liệu có thể không giúp được gì nhiều. Bạn nên tập trung vào các thay đổi khác. Chắc chắn rằng việc tăng số mẫu cho tập huấn luyện chỉ khiến quá trình huấn luyện mô hình trên tập này trở nên khó khăn hơn. (Lý do sẽ được giải thích trong chương sau.)

Nếu tỉ lệ lỗi trên tập huấn luyện là 15% (tức độ chính xác 85%), nhưng mục tiêu của bạn là 5% lỗi (độ chính xác 95%), thì vấn đề trước tiên cần giải quyết là cải thiện chất lượng thuật toán trên tập huấn luyện. Chất lượng trên tập phát triển/kiểm tra thường thấp hơn trên tập huấn luyện. Bởi vậy, nếu bạn đang có độ chính xác 85% trên các mẫu mà thuật toán từng thấy, thì không thể nào đạt được độ chính xác 95% cho các mẫu mà thuật toán chưa thấy bao giờ.

Giả sử như trên rằng thuật toán của bạn có 16% lỗi (độ chính xác 84%) trên tập phát triển. Chúng ta tách 16% lỗi này ra hai thành phần:

- Thứ nhất, tỉ lệ lỗi của thuật toán trên tập huấn luyện, là 15% trong ví dụ này. Chúng ta tạm coi giá trị này như **độ chệch** của thuật toán.
- Thứ hai, chất lượng của thuật toán trên tập phát triển (hoặc kiểm tra) kém hơn bao nhiêu so với trên tập huấn luyện. Trong ví dụ này, thuật toán làm việc kém hơn 1% trên tập phát triển so với tập huấn luyện. Chúng ta tạm coi giá trị này như **phương sai** của thuật toán [6].

Một số thay đổi trong thuật toán học có thể giải quyết thành phần thứ nhất của lỗi -- **độ chệch** -- và cải thiện chất lượng của nó trên tập huấn luyện. Một số thay đổi giải quyết thành phần thứ hai -- **phương sai** -- và giúp thuật toán tổng quát hóa tốt hơn từ tập huấn luyện tới tập phát triển/kiểm tra [7]. Việc nắm được thành phần nào trong hai thành phần lỗi trên là đáng giải quyết hơn sẽ rất hữu ích để trong việc chọn ra những thay đổi tiềm năng nhất.

Phát triển trực giác tốt về Độ chệch và Phương sai sẽ giúp bạn chọn những thay đổi hữu hiệu cho thuật toán.

---

[6] Ngành thống kê có những định nghĩa chính thống hơn cho độ chệch và phương sai mà ở đây chúng ta không cần lưu tâm. Đại khái, độ chệch là tỉ lệ lỗi của thuật toán trên tập huấn luyện khi tập này rất lớn. Phương sai là độ giảm chất lượng trên tập kiểm tra so với tập huấn luyện trong thiết lập này. Khi phép đo lỗi là sai số toàn phương trung bình, bạn có thể viết ra công thức tính hai đại lượng này và chứng minh được rằng Tổng Lỗi = Độ Chệch + Phương Sai. Nhưng với mục đích xác định làm thế nào để tạo sự tiến triển trong một bài toán học máy, thì định nghĩa ít chính thống hơn của độ chệch và phương sai như trình bày ở đây là đã đủ.

[7] Cũng có một vài phương pháp có thể đồng thời giảm độ chệch và phương sai bằng các thay đổi lớn trong kiến trúc hệ thống. Tuy nhiên, những phương pháp này có xu hướng khó phát hiện và triển khai hơn.

21. Những ví dụ về Độ chệch và Phương sai

Hãy xem xét việc phân loại mèo của chúng ta. Một bộ phân loại "lý tưởng" (như con người) có thể đạt được hiệu suất gần như hoàn hảo cho việc này.

Giả sử thuật toán của bạn đạt được:

- Lỗi huấn luyện = 1%
- Lỗi phát triển = 11%

Vậy vấn đề gặp phải là gì? Áp dụng các định nghĩa từ những chương trước, chúng ta ước tính độ chệch là 1% và phương sai là 10% (=11%-1%). Do đó, thuật toán có **phương sai cao**. Tuy bộ phân loại có lỗi huấn luyện rất thấp, nhưng nó lại không khái quát hoá được cho tập phát triển. Hiện tượng này cũng được gọi là **quá khớp**.

Bây giờ hãy xem xét trường hợp sau:

- Lỗi huấn luyện = 15%
- Lỗi phát triển = 16%

Chúng ta ước tính độ chệch là 15% và phương sai là 1%. Bộ phân loại này khớp kém với tập huấn luyện với 15% lỗi nhưng lỗi ở tập phát triển chỉ cao hơn một chút so với tập huấn luyện. Do đó, bộ phân loại này có **độ chệch cao** nhưng phương sai thấp. Chúng ta nói thuật toán này đang **dưới khớp** (*underfit*).

Bây giờ hãy xem xét trường hợp sau:

- Lỗi huấn luyện = 15%
- Lỗi phát triển = 30%

Chúng ta ước tính độ chệch là 15% và phương sai là 15%. Bộ phân loại này có**độ chệch cao và phương sai cao**: Nó hoạt động kém ở tập huấn luyện, do đó có độ chệch cao, và chất lượng của nó trên tập phát triển còn tệ hơn, do đó nó cũng có phương sai cao. Thuật ngữ quá khớp/dưới khớp rất khó áp dụng ở đây vì bộ phân loại đồng thời bị quá khớp và dưới khớp.

Cuối cùng, hãy xem xét điều này:

- Lỗi huấn luyện = 0,5%
- Lỗi phát triển = 1%

Bộ phân loại này đang hoạt động tốt vì nó có độ chệch thấp và phương sai thấp. Chúc mừng bạn đã đạt được một hiệu suất tuyệt vời!

22. So sánh với tỉ lệ lỗi tối ưu

Trong ví dụ nhận dạng mèo của chúng ta, tỉ lệ lỗi "lý tưởng" -- tỉ lệ có thể đạt được bởi một bộ phân loại "tối ưu" -- là gần với 0%. Gần như mọi lúc, một người nhìn vào bức ảnh có thể nhận ra có mèo trong đó hay không; do đó chúng ta có thể hy vọng máy móc cũng làm được điều tương tự.

Một số bài toán khác thì khó hơn. Ví dụ như chúng ta xây dựng một hệ thống nhận dạng giọng nói và nhận ra rằng 14% các đoạn âm thanh có quá nhiều nhiễu nền hoặc khó hiểu tới mức ngay cả con người cũng không thể nghe được những gì được nói. Trong trường hợp này, ngay cả hệ thống "tối ưu" nhất cũng có thể có lỗi khoảng 14%.

Giả sử rằng với bài toán nhận dạng giọng nói này, thuật toán của bạn đạt được:

- Lỗi huấn luyện = 15%
- Lỗi phát triển = 30%

Chất lượng trên tập huấn luyện hiện đã gần với tỉ lệ lỗi tối ưu là 14%. Do đó, không có nhiều chỗ để cải thiện độ chệch hoặc chất lượng trên tập huấn luyện. Tuy nhiên, thuật toán này không tổng quát hoá tốt trên tập phát triển; do đó có rất nhiều chỗ để cải thiện lỗi do phương sai.

Ví dụ này tương tự như ví dụ thứ ba trong chương trước, cũng có lỗi huấn luyện là 15% và lỗi phát triển là 30%. Nếu tỉ lệ lỗi tối ưu là xấp xỉ 0%, thì 15% lỗi huấn luyện để lại nhiều khả năng cải thiện. Điều này gợi ý rằng những thay đổi làm giảm độ chệch có thể mang lại nhiều kết quả. Nhưng nếu tỉ lệ lỗi tối ưu là 14%, thì chất lượng tương tự trên tập huấn luyện cho thấy có rất ít cơ hội để cải thiện độ chệch của bộ phân loại.

Với các bài toán trong đó tỉ lệ lỗi tối ưu lớn hơn nhiều mức 0%, thì có một cách phân tích chi tiết hơn về lỗi của thuật toán. Tiếp tục với ví dụ nhận dạng giọng nói ở trên, lỗi tổng cộng trên tập phát triển là 30% có thể được chia nhỏ như sau (phân tích tương tự có thể áp dụng cho lỗi trên tập kiểm tra):

- **Tỉ lệ lỗi tối ưu ("độ chệch không tránh được")**: 14%. Giả sử chúng ra quyết định rằng, ngay cả khi dùng hệ thống nhận dạng giọng nói tốt nhất trên thế giới, thì ta vẫn phải chịu 14% lỗi. Ta có thể coi lỗi đó là phần "không tránh được" trong độ chệch của thuật toán học máy.
- **Độ chệch tránh được**: 1%. Hiệu giữa lỗi huấn luyện và lỗi tối ưu. [8]
- **Phương sai**: 15%. Hiệu giữa lỗi trên tập phát triển và lỗi trên tập huấn luyện.

Từ những định nghĩa trước, thì mối liên hệ giữa Độ chệch và Độ chệch tránh được là: [9]

Độ chệch = Tỉ lệ lỗi tối ưu ("độ chệch không tránh được") + Độ chệch tránh được

"Độ chệch tránh được" phản ánh thuật toán của bạn hoạt động kém hơn bao nhiêu

so với "bộ phân loại tối ưu".

Khái niệm phương sai vẫn giữ nguyên như trước. Theo lý thuyết, chúng ta luôn có thể giảm phương sai về gần bằng không bằng cách huấn luyện trên một tập huấn luyện cực lớn. Do đó, tất cả phương sai là "tránh được" khi tập dữ liệu đủ lớn và không có cái gọi là "phương sai không tránh được".

Xem xét thêm một ví dụ nữa, trong đó tỉ lệ lỗi tối ưu là 14%, ta có:

- Lỗi huấn luyện = 15%
- Lỗi phát triển = 16%

Trong khi ở chương trước chúng ta gọi đây là một bộ phân loại có độ chệch cao, bây giờ chúng ta nói rằng lỗi từ độ chệch tránh được là 1% và lỗi từ phương sai là khoảng 1%. Do đó, thuật toán của chúng ta đã là rất tốt và có rất ít khoảng trống để cải thiện. Nó chỉ kém đúng 2% so với tỉ lệ lỗi tối ưu.

Từ những ví dụ này chúng ta thấy rằng tỉ lệ lỗi tối ưu rất hữu ích cho việc định hướng các bước tiếp theo. Trong thống kê, tỉ lệ lỗi tối ưu còn được gọi là **tỉ lệ lỗi Bayes** hay tỉ lệ Bayes.

Làm sao chúng ta biết được tỉ lệ lỗi tối ưu? Với những việc mà con người làm tốt, như nhận dạng ảnh hay phiên thoại các đoạn âm thanh, bạn có thể nhờ ai đó gán nhãn sau đó đo độ chính xác của những nhãn này với tập huấn luyện. Điều này sẽ cung cấp một con số ước tính của tỉ lệ lỗi tối ưu. Nếu bạn làm việc với một bài toán mà ngay cả con người cũng khó giải (ví dụ như dự đoán xem nên gợi ý bộ phim nào, hay hiện quảng cáo nào trước người dùng), thì có thể sẽ khó để ước tính được tỉ lệ lỗi tối ưu.

Trong phần "So sánh với chất lượng mức con người (chương 33 tới chương 35)", tôi sẽ thảo luận chi tiết hơn quá trình so sánh chất lượng một thuật toán học máy với chất lượng mức con người.

Trong một vài chương trước, bạn đã học cách tính phương sai và độ chệch tránh được/không tránh được bằng cách xem xét tỉ lệ lỗi huấn luyện và tỉ lệ lỗi phát triển. Chương tiếp theo sẽ thảo luận về cách bạn có thể sử dụng những hiểu biết sâu sắc từ phân tích đó để ưu tiên các kỹ thuật làm giảm độ chệch hoặc các kỹ thuật làm giảm phương sai. Có nhiều kỹ thuật khác nhau nên áp dụng tuỳ thuộc vào vấn đề hiện tại trong dự án của bạn là độ chệch (tránh được) cao hay phương sai cao. Hãy đọc tiếp!

CHÚ THÍCH:

[8] Nếu con số này là âm, bạn đang làm tốt hơn ở trên tập huấn luyện so với tỉ lệ lỗi tối ưu. Điều này có nghĩa là bạn đang quá khớp tập huấn luyện và thuật toán đã ghi nhớ quá mức tập huấn luyện. Bạn nên tập trung vào các phương pháp giảm phương sai hơn là các phương pháp giảm độ chệch khác.

[9] Những định nghĩa này được chọn để truyền đạt cái nhìn sâu sắc về cách cải thiện thuật toán học máy của bạn. Những định nghĩa này khác với cách các nhà thống kê

định nghĩa Độ chệch và Phương sai. Về mặt kỹ thuật, những gì tôi định nghĩa là "Độ chệch" nên được gọi là "Lỗi chúng ta quy cho độ chệch" và "Độ chệch tránh được" nên là "Lỗi chúng ta quy cho độ chệch của thuật toán học mà lớn hơn tỉ lệ lỗi tối ưu" .

23. Xử lý Độ chệch và Phương sai

Đây là công thức đơn giản nhất để giải quyết các vấn đề độ chệch và phương sai:

- Nếu bạn có độ chệch tránh được cao, hãy tăng kích thước mô hình (ví dụ: tăng kích thước của mạng nơ-ron bằng cách thêm các tầng/nơ-ron).
- Nếu bạn có phương sai cao, hãy thêm dữ liệu vào tập huấn luyện.

Nếu bạn có thể tăng kích thước của mạng nơ-ron và dữ liệu huấn luyện lên vô hạn, thì bạn sẽ có khả năng xử lý rất tốt trên nhiều bài toán học máy.

Trong thực tế, việc tăng kích thước của mô hình cuối cùng sẽ khiến bạn gặp phải các vấn đề về tính toán bởi vì việc huấn luyện các mô hình cực lớn là rất chậm. Bạn có thể cũng sẽ làm cạn kiệt khả năng có được nhiều dữ liệu huấn luyện hơn. (Ngay cả trên mạng internet cũng chỉ có một số lượng hữu hạn hình ảnh mèo!)

Những kiến trúc mô hình khác nhau -- ví dụ, các kiến trúc mạng nơ-ron khác nhau -- sẽ có các mức độ chệch/phương sai khác nhau cho vấn đề của bạn. Những nghiên cứu gần đây về học sâu đã phát triển nhiều kiến trúc mô hình đột phá. Vì vậy, nếu bạn đang sử dụng mạng nơ-ron, những tài liệu học thuật có thể là một nguồn cảm hứng tuyệt vời. Ngoài ra còn có rất nhiều cài đặt mã nguồn mở tuyệt vời trên GitHub. Nhưng kết quả của việc thử nghiệm các kiến trúc mới khó dự đoán hơn so với công thức đơn giản của việc tăng kích thước mô hình và thêm dữ liệu.

Nhìn chung, việc tăng kích thuớc mô hình làm giảm độ chệch, nhưng nó cũng có thể làm tăng phương sai và tăng nguy cơ quá khớp. Tuy nhiên, vấn đề quá khớp này thường chỉ phát sinh khi bạn không sử dụng điều chuẩn (*regularization*). Nếu bạn thêm vào một phương pháp điều chuẩn được thiết kế tốt, thì bạn thường có thể tăng kích thuớc mô hình một cách an toàn mà không tăng quá khớp.

Giả sử bạn đang áp dụng học sâu, với điều chuẩn L2 hoặc dropout, với tham số điều chuẩn hoạt động tốt nhất tập phát triển. Nếu bạn tăng kích thuớc mô hình, thường thì chất lượng của mô hình sẽ giữ nguyên hoặc cải thiện; nó thường không có khả năng xấu đi đáng kể. Lý do duy nhất của việc tránh sử dụng một mô hình lớn hơn là phần chi phí tính toán tăng thêm.

24. Đánh đổi giữa Độ chệch và Phương sai

Bạn có thể đã nghe về "sự đánh đổi giữa Độ chệch và Phương sai". Trong các thay đổi khả dĩ đối với hầu hết các thuật toán học, có một số phương pháp giúp giảm sai số độ chệch nhưng với phải trả giá bằng việc tăng phương sai và ngược lại. Đây là sự "đánh đổi" giữa độ chệch và phương sai.

Ví dụ việc tăng kích thước mô hình -- như thêm các nơ-ron/tầng trong mạng nơ-ron hoặc thêm các đầu vào đặc trưng -- nhìn chung sẽ giảm độ chệch nhưng có thể làm tăng phương sai. Ngoài ra, việc thêm điều chuẩn thường làm tăng độ chệch nhưng giảm phương sai.

Ngày nay, chúng ta có thể truy cập vào nguồn dữ liệu phong phú và có thể sử dụng các mạng nơ-ron rất lớn (trong học sâu). Vì thế mà ta ít phải đánh đổi hơn. Hiện có nhiều lựa chọn hơn để giảm độ chệch mà không làm ảnh hưởng phương sai và ngược lại.

Ví dụ, bạn thường có thể tăng kích thước mạng nơ-ron và điều chỉnh phương thức điều chuẩn để giảm độ chệch mà không làm tăng đáng kể phương sai. Bằng cách thêm dữ liệu huấn luyện, bạn cũng thường có thể giảm phương sai mà không ảnh hưởng đến độ chệch.

Nếu bạn chọn một kiến trúc mô hình phù hợp với tác vụ của mình, bạn cũng có thể giảm đồng thời độ chệch và phương sai. Tuy nhiên sẽ rất khó để tìm ra một kiến trúc như vậy.

Trong một vài chương tới, chúng ta sẽ thảo luận thêm về các kỹ thuật cụ thể để giải quyết các vấn đề liên quan tới độ chệch và phương sai.

25. Kỹ thuật giảm độ chệch có thể tránh được

Nếu thuật toán học gặp vấn đề với *độ chệch có thể tránh được* lớn, bạn có thể thử những kỹ thuật sau:

- **Tăng kích thước mô hình** (ví dụ như số lượng nơ-ron/tầng): Kĩ thuật này giúp giảm độ chệch vì nó cho phép khớp (fit) tập huấn luyện tốt hơn. Nếu thấy việc này làm tăng phương sai, bạn hãy dùng điều chuẩn (*regularization*) -- thường dùng để giảm trừ việc tăng phương sai.
- **Thay đổi các đặc trưng đầu vào dựa trên những nhận định từ phân tích lỗi**: Giả sử việc phân tích lỗi gợi ý rằng nên tạo thêm các đặc trưng bổ sung nhằm giúp thuật toán loại bỏ một nhóm các lỗi đặc thù. (Chúng ta sẽ bàn vấn đề này chi tiết hơn ở chương sau.) Những đặc trưng mới này có thể hiệu quả với cả độ chệch và phương sai. Theo lý thuyết, thêm đặc trưng có thể làm tăng phương sai; tuy nhiên nếu thấy đúng là phương sai bị tăng, thì bạn hãy dùng điều chuẩn -- thường dùng để loại bỏ việc tăng phương sai.
- **Giảm hoặc loại bỏ điều chuẩn** (điều chuẩn L2, điều chuẩn L1, dropout): Việc này sẽ làm giảm độ chệch có thể tránh được, nhưng sẽ đồng thời làm tăng phương sai.
- **Thay đổi kiến trúc mô hình** (ví dụ như kiến trúc mạng nơ-ron) để phù hợp hơn với bài toán của bạn. Lưu ý rằng, kỹ thuật này có thể tác động đến cả độ chệch và phương sai.

Một phương pháp không hữu ích:

- **Thêm dữ liệu huấn luyện**: Kỹ thuật này hữu ích với các vấn đề về phương sai nhưng thường không có tác động đáng kể đến độ chệch.

26. Phân tích lỗi trên tập huấn luyện

Thuật toán của bạn phải hoạt động tốt trên tập huấn luyện trước khi bạn có thể mong đợi nó hoạt động tốt trên tập phát triển/kiểm tra.

Ngoài các kỹ thuật giải quyết độ chệch cao được mô tả trước đây, đôi khi tôi cũng thực hiện phân tích lỗi trên *dữ liệu huấn luyện*, theo một quá trình tương tự như phân tích lỗi trên tập phát triển Eyeball. Điều này có thể hữu ích nếu thuật toán của bạn có độ chệch cao, ví dụ như nếu nó không khớp tốt với tập huấn luyện.

Ví dụ: giả sử bạn đang xây dựng một hệ thống nhận dạng giọng nói cho một ứng dụng nào đó và đã thu thập một tập huấn luyện gồm nhiều đoạn âm thanh từ các tình nguyện viên. Nếu hệ thống không hoạt động tốt trên tập huấn luyện, bạn có thể xem xét việc nghe thử một bộ khoảng 100 mẫu mà thuật toán hoạt động kém để hiểu các hạng mục lỗi chính trên tập huấn luyện. Tương tự như phân tích lỗi trên tập phát triển, bạn có thể đếm các lỗi theo các hạng mục như sau:

Đoạn âm thanh	Nhiễu nền lớn	Đoạn nói nhanh	Mic ở xa	Chi tiết thêm
1	✔			Tiếng ồn xe hơi
2	✔		✔	Tiếng ồn nhà hàng
3		✔	✔	Người dùng la hét khắp phòng khách?
4	✔			Quán cà phê
% tổng	75%	25%	50%	

Trong ví dụ này, bạn có thể nhận ra rằng thuật toán đang gặp khó khăn với các mẫu huấn luyện có nhiều nhiễu nền. Do đó, bạn có thể tập trung vào các kỹ thuật cho phép thuật toán khớp hơn với các mẫu huấn luyện có nhiễu nền.

Bạn cũng có thể kiểm tra kỹ xem, liệu một người có thể diễn dịch với cùng các đoạn âm thanh đầu vào cho thuật toán học hay không. Nếu có quá nhiều nhiễu nền, nhiều đến nỗi không ai có thể nghe ra những gì được nói, thì có thể không hợp lý khi hy vọng bất kỳ thuật toán nào có thể nhận ra chính xác những phát ngôn đó. Chúng ta sẽ thảo luận về lợi ích của việc so sánh thuật toán với chất lượng mức con người trong phần sau.

27. Các kỹ thuật giảm phương sai

Nếu như thuật toán có phương sai lớn, bạn có thể thử các kỹ thuật sau:

- **Thêm dữ liệu huấn luyện**: Đây là cách đơn giản và đáng tin cậy nhất để giảm phương sai, miễn là bạn có thể lấy dữ liệu nhiều hơn đáng kể và đủ năng lực tính toán để xử lý dữ liệu.
- **Thêm điều chuẩn** (điều chuẩn L2, điều chuẩn L1, dropout): Kỹ thuật này làm giảm phương sai nhưng cũng làm tăng độ chệch.
- **Thêm kỹ thuật dừng sớm** (nghĩa là dừng sớm quá trình hạ gradient, dựa vào lỗi trên tập phát triển): Kỹ thuật này giúp giảm phương sai nhưng làm tăng độ chệch. Kỹ thuật dừng sớm hoạt động rất giống các phương pháp điều chuẩn và một số tác giả cũng coi nó là một kỹ thuật điều chuẩn.
- **Chọn đặc trưng để giảm số lượng/kiểu đặc trưng đầu vào**: Kỹ thuật này có thể giúp giải quyết các vấn đề về phương sai, nhưng nó cũng có thể làm tăng độ chệch. Việc giảm một ít số lượng các đặc trưng (giả sử từ 1.000 xuống 900 đặc trưng) dường như không có ảnh hưởng lớn đến độ chệch. Việc giảm đáng kể số đặc trưng (giả sử từ 1.000 xuống còn 100 đặc trưng, hay giảm 10 lần) nhiều khả năng mang lại tác dụng đáng kể, miễn là bạn không loại trừ quá nhiều các đặc trưng hữu ích. Trong học sâu hiện đại, khi dữ liệu dồi dào, đã có những thay đổi từ việc lựa chọn đặc trưng. Giờ đây, hầu như chúng ta dùng tất cả các đặc trưng hiện có cho thuật toán và để nó tự chọn ra những đặc trưng sẽ sử dụng dựa trên chính tập dữ liệu đó. Nhưng khi tập huấn luyện nhỏ, kỹ thuật lựa chọn đặc trưng vẫn có thể rất hữu ích.
- **Giảm kích thước mô hình** (chẳng hạn như số lượng nơ-ron/tầng): *Sử dụng một cách thận trọng*. Kỹ thuật này có thể làm giảm phương sai, trong khi có thể làm tăng độ chệch. Tuy nhiên, tôi không khuyến khích sử dụng kỹ thuật này để giảm phương sai. Thêm điều chuẩn thường cho chất lượng phân loại tốt hơn. Ưu điểm của việc giảm kích thước mô hình là giảm chi phí tính toán và do đó tăng tốc độ huấn luyện mô hình. Nếu việc tăng tốc độ huấn luyện mô hình là hữu ích, thì hãy xem xét việc giảm kích thước mô hình bằng mọi cách. Nhưng nếu mục tiêu là giảm phương sai và bạn không quan tâm đến chi phí tính toán, thì thay vào đó bạn hãy xem xét việc thêm điều chuẩn.

Dưới đây là hai chiến thuật bổ sung, được lặp lại từ chương trước về giải quyết độ chệch:

- **Thay đổi các đặc trưng đầu vào dựa trên nhận định từ phân tích lỗi**: Giả sử việc phân tích lỗi gợi ý rằng nên tạo thêm các đặc trưng bổ sung nhằm giúp thuật toán loại bỏ một nhóm các lỗi đặc thù. Những đặc trưng mới này có thể hiệu quả với cả độ chệch và phương sai. Theo lý thuyết, thêm đặc trưng có thể làm tăng phương sai; nhưng nếu bạn thấy đúng là phương sai bị tăng, thì hãy sử dụng điều chuẩn -- thường dùng để giảm trừ việc tăng phương sai.

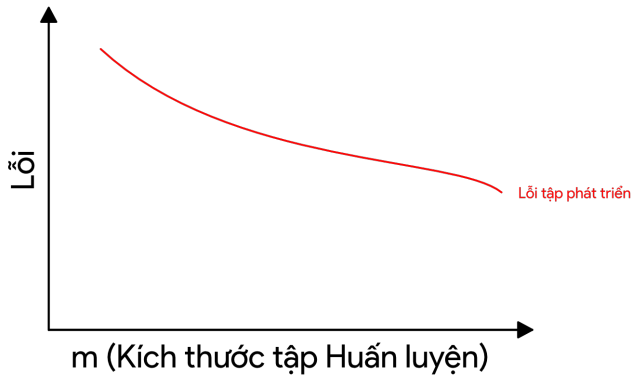
- **Thay đổi kiến trúc mô hình** (ví dụ như kiến trúc mạng nơ-ron) để phù hợp hơn với bài toán của bạn: Kỹ thuật này có thể tác động đến cả độ lệch và phương sai.

Phần 4: Đồ thị quá trình học

28. Chẩn đoán độ chệch và phương sai: Đồ thị quá trình học

Chúng ta đã xem qua một số cách ước tính số lượng lỗi tạo ra bởi độ chệch tránh được so với phương sai. Một trong số đó là ước lượng tỉ lệ lỗi tối ưu và tính toán lỗi của thuật toán trên tập huấn luyện và tập phát triển. Hãy cũng thảo luận một kỹ thuật khác thậm chí còn mang lại nhiều thông tin hơn: biểu diễn một đồ thị quá trình học.

Một đồ thị quá trình học cho thấy sự tương quan giữa lỗi của tập phát triển so với số lượng các mẫu huấn luyện. Để biểu diễn nó, bạn cần áp dụng thuật toán của bạn với các tập huấn luyện có độ lớn khác nhau. Ví dụ, nếu bạn có 1.000 mẫu, bạn có thể huấn luyện riêng biệt các bản sao của thuật toán trên các tập 100, 200, 300, ..., 1.000 mẫu. Sau đó bạn có thể biểu diễn sự thay đổi giữa lỗi của tập phát triển so với độ lớn của tập huấn luyện. Dưới đây là một ví dụ:

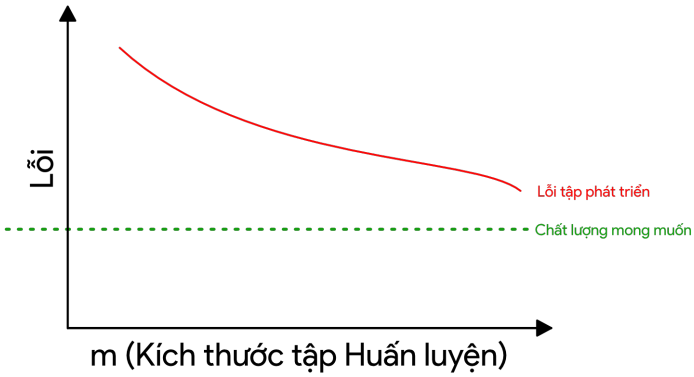


Khi kích thước tập huấn luyện tăng, lỗi của tập phát triển nên giảm.

Chúng ta thường sẽ có một số "tỉ lệ lỗi mong muốn" mà chúng ta hy vọng thuật toán của mình cuối cùng sẽ đạt được. Ví dụ:

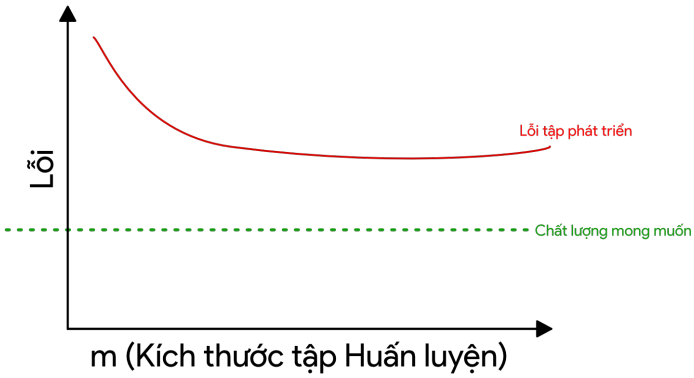
- Nếu chúng ta hy vọng đạt được chất lượng ở cấp độ con người, thì tỷ lệ lỗi của con người là "tỉ lệ lỗi mong muốn".
- Nếu thuật toán học của chúng ta được dùng trong một sản phẩm nào đó (ví dụ như cung cấp ảnh mèo), chúng ta có thể có một trực giác về mức chất lượng cần thiết để người dùng có được trải nghiệm tốt nhất.
- Nếu bạn đã làm việc trên một ứng dụng quan trọng trong thời gian dài, thì bạn sẽ có trực giác về mức cải thiện hợp lý có thể đạt được trong quý/năm tới.

Thêm mức chất lượng mong muốn vào đồ thị quá trình học của bạn:



Bạn có thể ngoại suy đường cong "lỗi tập phát triển" (màu đỏ) để ước tính mức độ cải thiện có thể đạt được so với mức chất lượng mong muốn bằng cách thêm dữ liệu vào. Ví dụ trên cho thấy bạn có thể đạt được mức chất lượng mong muốn bằng cách tăng gấp đôi độ lớn tập huấn luyện.

Tuy nhiên nếu đường cong lỗi tập phát triển đã "nằm ngang" (phẳng), thì bạn có thể hiểu ngay lập tức rằng việc thêm vào dữ liệu cũng sẽ không giúp bạn đạt được mục tiêu:



Do đó nhìn vào đồ thị đường cong học tập có thể giúp bạn tránh khỏi việc dành hàng tháng trời thu thập một lượng dữ liệu lớn gấp đôi, chỉ để nhận ra rằng điều đó là vô ích.

Một nhược điểm của quá trình này là nếu bạn chỉ nhìn vào đường cong lỗi của tập phát triển, thì có thể bạn sẽ khó ngoại suy và dự đoán chính xác vị trí đường cong đỏ khi có thêm dữ liệu. Một đồ thị khác có thể giúp bạn dự đoán sự tác động của việc thêm dữ liệu đó là: đồ thị lỗi tập huấn luyện.

29. Vẽ đồ thị sai số huấn luyện

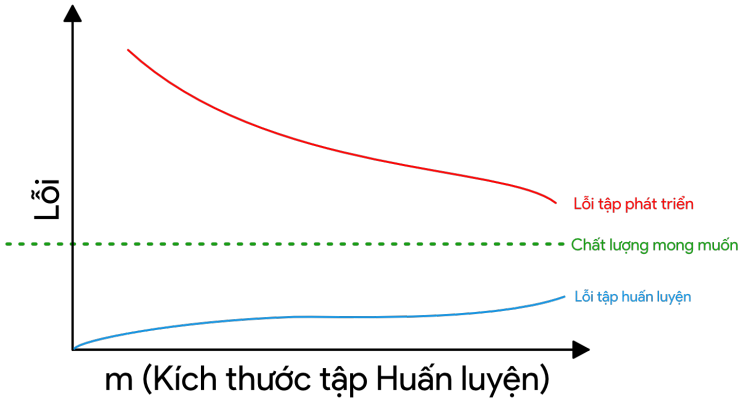
Sai số tập phát triển (và tập kiểm tra) thường giảm khi kích thước tập huấn luyện tăng lên. Nhưng sai số tập huấn luyện lại thường xuyên *tăng* khi kích thước tập huấn luyện tăng.

Chúng ta hãy mô tả ảnh hưởng này bằng một ví dụ. Giả sử tập huấn luyện chỉ gồm 2 mẫu: Một ảnh mèo và một ảnh không phải mèo. Khi đó thuật toán học máy có thể dễ dàng ghi nhớ cả 2 mẫu trong tập huấn luyện, và cho 0% sai số huấn luyện. Thậm chí nếu cả 2 mẫu huấn luyện bị gán nhãn sai, thuật toán vẫn ghi nhớ dễ dàng cả 2 nhãn.

Bây giờ giả định tập huấn luyện có 100 mẫu. Một vài mẫu thậm chí bị gán nhãn sai, hoặc một vài hình ảnh thì không rõ ràng do bị mờ, nên ngay cả con người cũng không thể khẳng định đó là một chú mèo. Có lẽ thuật toán học vẫn có thể "ghi nhớ" được hầu hết tập huấn luyện, nhưng nó khó mà đạt được 100% độ chính xác vào lúc này. Bằng cách gia tăng tập huấn luyện từ 2 lên 100 mẫu, bạn sẽ nhận ra rằng độ chính xác của tập huấn luyện sẽ giảm một ít.

Cuối cùng, giả sử tập huấn luyện có 10.000 mẫu. Trong trường hợp này, sẽ khó hơn cho thuật toán khớp hoàn hảo 10.000 mẫu, đặc biệt là nếu có một vài mẫu không rõ ràng hoặc bị gán sai nhãn. Do đó, trên tập huấn luyện thuật toán của bạn sẽ hoạt động càng kém hơn trước.

Chúng ta hãy thêm một đồ thị sai số huấn luyện vào các hình trước đó:

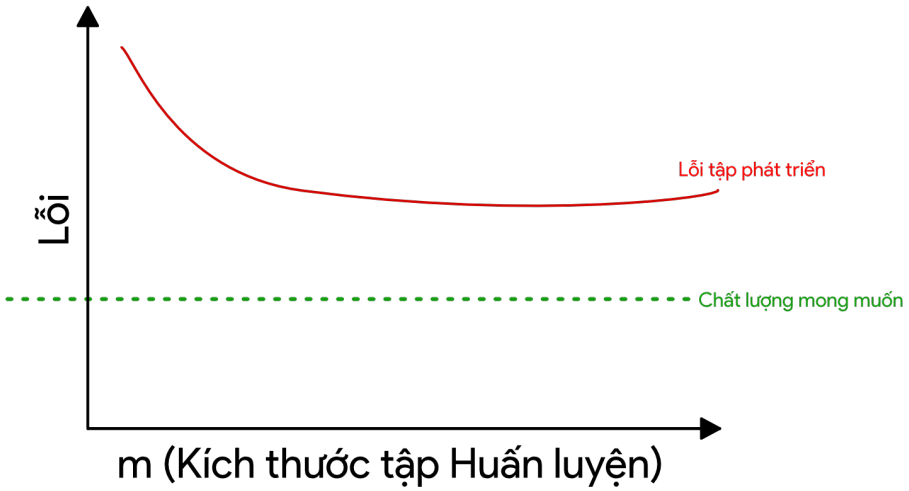


Bạn có thể thấy rằng đồ thị "sai số huấn luyện" (màu xanh lam) tăng theo kích thước của tập huấn luyện. Thêm nữa, thuật toán của bạn thường hoạt động tốt trên tập huấn luyện hơn là tập phát triển; do đó đồ thị sai số tập phát triển (màu đỏ) hoàn toàn nằm trên đồ thị sai số huấn luyện.

Tiếp theo chúng ta sẽ thảo luận làm thế nào để diễn giải những đồ thị này.

30. Diễn giải đồ thị quá trình học: Độ chệch cao

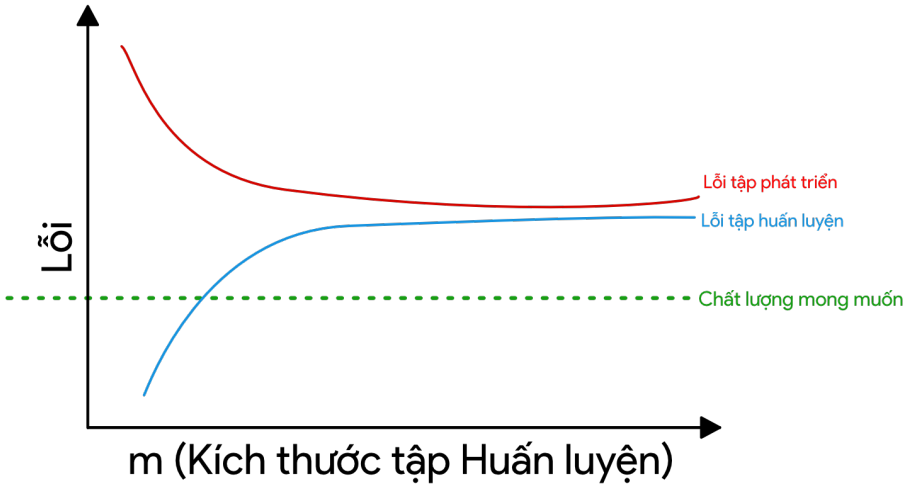
Giả sử đường cong sai số trên tập phát triển có dạng như sau:



Như đã thảo luận, nếu đường cong sai số trên tập phát triển đã nằm ngang, việc chỉ thêm dữ liệu sẽ khó có thể đem về hiệu suất ta mong muốn.

Tuy nhiên, sẽ thật khó để ngoại suy chính xác đường cong sai số trên tập phát triển (màu đỏ) sẽ trông như thế nào. Trong trường hợp tập phát triển nhỏ, việc dự đoán chính xác sẽ càng trở nên khó khăn bởi khi đó đường cong này sẽ có khả năng bị nhiễu.

Giả sử chúng ta thêm đường cong sai số tập huấn luyện vào biểu đồ như hình dưới:



Lúc này, bạn có thể hoàn toàn chắc chắn việc chỉ thêm dữ liệu là không đủ. Tại sao vậy? Hãy nhớ hai nhận định sau:

Khi chúng ta thêm dữ liệu huấn luyện, sai số huấn luyện chỉ có thể tăng lên. Vì vậy, đường cong chỉ sai số huấn luyện (màu xanh lam) chỉ có thể giữ nguyên hoặc hướng cao lên. Bởi vậy, đường cong đó chỉ có thể cách xa hơn hiệu suất mong đợi (màu xanh lục).

Đường cong sai số tập phát triển (màu đỏ) thường cao hơn so với đường cong sai số tập huấn luyện (màu xanh lam). Vì vậy, việc lấy thêm dữ liệu không thể nào giảm đường cong sai số tập phát triển xuống mức hiệu suất mong muốn khi ngay cả sai số trên tập huấn luyện vẫn còn lớn hơn mức đó.

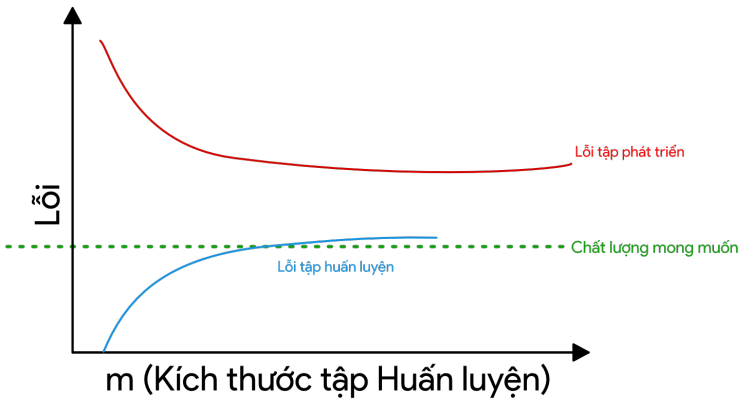
Đánh giá cả đường cong sai số tập phát triển lẫn đường cong sai số tập huấn luyện trên cùng một biểu đồ giúp những ngoại suy về đường cong sai số tập phát triển có độ tin cậy cao hơn.

Để dễ thảo luận, giả sử hiệu suất mong muốn chính là ước lượng của tỉ lệ lỗi tối ưu. Đồ thị trên trở thành ví dụ chuẩn "sách giáo khoa" về hình dáng của một đồ thị quá trình học với độ chệch có thể tránh lớn: Khi tập huấn luyện có kích cỡ lớn nhất -- tương ứng với tất cả dữ liệu trong tập huấn luyện - có một khoảng cách lớn giữa sai số huấn luyện và hiệu suất mong muốn. Đây chính là dấu hiệu của độ chệch có thể tránh lớn. Ngược lại, lúc này, khoảng cách nhỏ giữa đường cong của tập huấn luyện và đường cong của tập phát triển tương ứng với phương sai nhỏ.

Trước kia, chúng ta chỉ đo sai số tập huấn luyện và sai số tập phát triển tại điểm ngoài cùng bên phải của đồ thị, tương ứng với việc sử dụng tất cả dữ liệu trong tập huấn luyện. Biểu diễn đầy đủ đồ thị quá trình học sẽ cho chúng ta một bức tranh tổng thể hơn về chất lượng của những thuật toán trên các kích cỡ tập huấn luyện khác nhau.

31. Giải nghĩa các đồ thị quá trình học: Những trường hợp khác

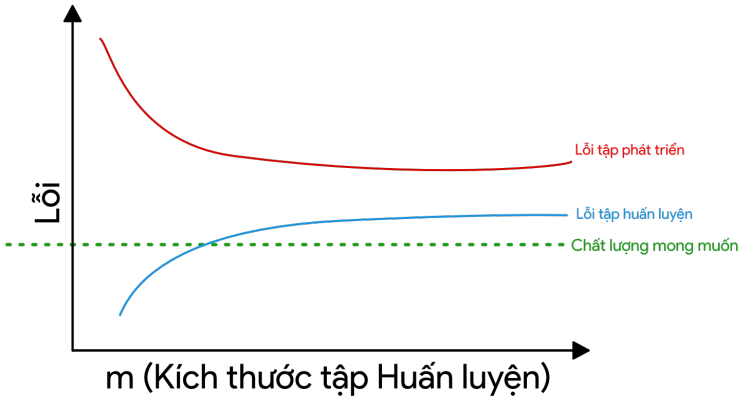
Hãy xem xét đồ thị quá trình học này:



Đồ thị này thể hiện độ chệch lớn, phương sai lớn hay cả hai?

Đường cong lỗi huấn luyện (màu xanh lam) tương đối thấp và đường cong lỗi phát triển (màu đỏ) cao hơn nhiều so với lỗi huấn luyện. Do đó, độ chệch nhỏ, nhưng phương sai lớn. Thêm dữ liệu huấn luyện có thể giúp thu hẹp khoảng cách giữa lỗi phát triển và lỗi huấn luyện.

Bây giờ, hãy xem xét đồ thị này:



Lần này, lỗi huấn luyện lớn, vì nó cao hơn nhiều so với mức chất lượng mong muốn. Lỗi phát triển cũng lớn hơn nhiều so với lỗi huấn luyện. Vì vậy, bạn có độ chệch đáng kể và phương sai cũng đáng kể. Bạn sẽ phải tìm cách giảm cả độ chệch và phương sai trong thuật toán của mình.

32. Vẽ đồ thị quá trình học

Giả sử bạn có một tập huấn luyện rất nhỏ gồm 100 mẫu. Bạn huấn luyện thuật toán của mình bằng cách sử dụng một tập hợp con được chọn ngẫu nhiên gồm 10 mẫu, sau đó 20 mẫu, sau đó 30, lên đến 100, tăng số lượng mẫu theo các khoảng mười. Sau đó, bạn sử dụng 10 điểm dữ liệu này để vẽ đồ thị quá trình học. Bạn có thể thấy rằng đồ thị trông hơi nhiễu (có nghĩa là các giá trị cao hơn/thấp hơn dự kiến) ở kích thước tập huấn luyện nhỏ hơn.

Khi huấn luyện chỉ với 10 mẫu được chọn ngẫu nhiên, bạn có thể không may mắn và có một tập huấn luyện đặc biệt "xấu", chẳng hạn như một tập có nhiều mẫu không rõ ràng/bị gán nhãn sai. Hoặc, bạn có thể gặp may mắn và nhận được một tập huấn luyện đặc biệt "tốt". Có một tập huấn luyện nhỏ đồng nghĩa với lỗi trên tập phát triển và tập huấn luyện có thể dao động ngẫu nhiên.

Nếu ứng dụng học máy của bạn bị lệch nhiều về một lớp: chẳng hạn như nhiệm vụ phân loại mèo mà số mẫu âm (không phải mèo) lớn hơn nhiều số mẫu dương (là mèo). Hoặc, nếu nó có một số lượng lớn các lớp (chẳng hạn như nhận dạng 100 loài động vật khác nhau), khi đó xác suất chọn một tập huấn luyện rất không "mang tính đại diện" hoặc xấu cũng lớn hơn. Ví dụ: nếu 80% mẫu của bạn là mẫu âm ($y = 0$), và chỉ 20% là mẫu dương ($y = 1$), thì có khả năng một tập huấn luyện gồm 10 mẫu chỉ chứa các mẫu âm, thì rất khó để thuật toán học được điều gì đó có ý nghĩa.

Nếu nhiễu trong đồ thị quá trình học khiến bạn khó nhìn thấy xu hướng thực sự, thì đây là hai giải pháp:

- Thay vì chỉ huấn luyện một mô hình trên 10 mẫu, hãy chọn ra ngẫu nhiên một vài (ví dụ 3-10) tập huấn luyện khác nhau gồm 10 mẫu bằng cách lấy mẫu có hoàn lại [10] từ bộ dữ liệu 100 mẫu ban đầu. Huấn luyện mô hình khác nhau trên mỗi tập đó và tính toán lỗi huấn luyện, lỗi phát triển của từng mô hình sau khi huấn luyện xong. Tính toán và vẽ đồ thị lỗi trung bình trên tập huấn luyện và lỗi trung bình trên tập phát triển.
- Nếu tập huấn luyện của bạn bị lệch về một lớp, hoặc nếu nó có nhiều lớp, hãy chọn một tập hợp con "cân bằng" thay vì chọn ngẫu nhiên 10 trên 100 mẫu huấn luyện. Ví dụ, bạn có thể chắc chắn rằng 2/10 các mẫu là các mẫu dương và 8/10 là âm. Tổng quát hơn, bạn có thể đảm bảo tỷ lệ các mẫu trong mỗi lớp càng gần với tỉ lệ trong tập huấn luyện ban đầu.

Nếu bạn đã thử vẽ các đồ thị quá trình học và kết luận rằng các đường cong quá nhiễu để nhìn thấy các xu hướng cơ bản, thì hãy sử dụng một trong những kỹ thuật trên. Nếu tập huấn luyện của bạn có quy mô lớn, ví dụ hơn 10.000 mẫu, và phân phối lớp của bạn không bị lệch nhiều, có lẽ bạn không cần các kỹ thuật này.

Cuối cùng, vẽ đồ thị quá trình học có thể tốn kém về mặt tính toán: Ví dụ: bạn có thể phải huấn luyện mười mô hình với 1.000, rồi 2.000, cho đến 10.000 mẫu. Huấn luyện các mô hình với các bộ dữ liệu nhỏ nhanh hơn nhiều so với huấn luyện các mô hình với các bộ dữ liệu lớn. Do đó, thay vì cách đều các kích thước tập huấn luyện

theo tỷ lệ tuyến tính như trên, bạn có thể huấn luyện các mô hình với 1.000, 2.000, 4.000, 6.000 và 10.000 mẫu. Điều này vẫn sẽ cung cấp cho bạn một cảm giác rõ ràng về các xu hướng trong các đồ thị quá trình học. Tất nhiên, kỹ thuật này chỉ thích hợp nếu chi phí tính toán để huấn luyện tất cả các mô hình bổ sung là đáng kể.

CHÚ THÍCH

[10] Ở đây, việc lấy mẫu *có hoàn lại* có nghĩa là: Bạn sẽ chọn ngẫu nhiên 10 mẫu khác nhau trong số 100 để tạo thành tập huấn luyện đầu tiên của mình. Sau đó để tạo tập huấn luyện thứ hai, bạn sẽ lại lấy 10 mẫu ngẫu nhiên trong 100 mẫu ban đầu. Vì vậy, có thể một mẫu cụ thể xuất hiện trong cả tập huấn luyện thứ nhất và thứ hai. Ngược lại, nếu bạn lấy mẫu *không hoàn lại*, tập huấn luyện thứ hai sẽ chỉ được chọn từ 90 mẫu không được chọn ở lần đầu tiên. Trong thực tế, sử dụng lấy mẫu có hoàn lại hoặc không hoàn lại không tạo ra sự khác biệt lớn, nhưng lấy mẫu có hoàn lại là cách làm phổ biến.

Phần 5: So sánh với chất lượng mức con người

33. Tại sao cần so sánh chất lượng mức con người?

Nhiều hệ thống học máy hướng tới tự động hóa những thứ con người làm tốt. Ví dụ như nhận dạng hình ảnh, nhận dạng giọng nói, và phân loại thư rác. Các thuật toán học cũng đã được cải thiện rất nhiều, đến mức vượt qua chất lượng mức con người trong ngày càng nhiều tác vụ.

Hơn nữa, có rất nhiều lý do khiến việc xây dựng một hệ thống học máy dễ dàng hơn nếu bạn đang giải quyết một tác vụ mà con người có thể làm tốt:

1. **Dễ dàng thu thập dữ liệu từ người gán nhãn**. Ví dụ, con người nhận diện hình ảnh mèo tốt, nên tất nhiên việc dùng người để gán nhãn sẽ cung cấp nhãn có độ chính xác cao cho thuật toán học.
2. **Phân tích lỗi có thể dựa vào trực giác của con người**. Giả sử rằng một thuật toán nhận dạng giọng nói hoạt động kém hơn so với con người. Giả dụ nó ghi nhầm một đoạn âm thanh thành "This recipe calls for a *pear* of apples," (dịch là "công thức nấu ăn này cần một quả lê của táo") với lỗi từ "pair" trở thành "pear". Bạn có thể dựa vào trực giác và cố gắng hiểu thông tin nào mà con người dùng để nghe ra được bản ghi thoại chuẩn và dùng thông tin này để điều chỉnh thuật toán.
3. **Sử dụng chất lượng mức con người để ước tính tỷ lệ lỗi tối ưu cũng như đặt ra một "tỷ lệ lỗi mong muốn."** Giả sử thuật toán của bạn trả về 10% lỗi trong một tác vụ, nhưng con người chỉ lỗi 2%. Dựa vào đó, chúng ta biết rằng tỷ lệ lỗi tối ưu bằng hoặc nhỏ hơn 2% và độ chệch có thể tránh được ở mức nhỏ nhất là 8%. Vì vậy, bạn nên thử các kỹ thuật giảm độ chệch.

Mặc dù mục số 3 dường như không quan trọng, tôi thấy rằng việc đặt mục tiêu về tỷ lệ lỗi ở mức hợp lý sẽ giúp đẩy nhanh tiến độ của nhóm. Việc biết thuật toán của bạn có độ chệch cao có thể tránh được là vô cùng có giá trị và mở ra nhiều tùy chọn để thử nghiệm.

Có những tác vụ mà ngay cả con người cũng làm không tốt. Ví dụ, chọn một cuốn sách để giới thiệu cho bạn; hoặc chọn một tin quảng cáo để hiển thị cho người dùng trên một trang web; hoặc dự đoán thị trường chứng khoán. Máy tính đã làm việc hiệu quả hơn hầu hết mọi người trong những tác vụ này. Với các ứng dụng này, chúng ta gặp phải các vấn đề sau:

- Khó khăn hơn khi gán nhãn. Ví dụ, người gán nhãn khó có thể dán nhãn một cơ sở dữ liệu người dùng cho tác vụ "tối ưu" việc gợi ý sách. Nếu đang vận hành một trang web hoặc ứng dụng bán sách, bạn có thể lấy dữ liệu bằng cách hiển thị sách cho người dùng và xem những gì họ mua. Nếu không vận hành một trang web như vậy, bạn cần tìm những cách sáng tạo hơn để lấy dữ liệu.
- Khó tin tưởng trực giác của con người. Ví dụ, gần như không ai có thể dự đoán được thị trường chứng khoán. Vì thế, nếu thuật toán dự đoán cổ phiếu của chúng

ta không tốt hơn dự đoán ngẫu nhiên, thì thật khó để tìm ra cách cải thiện nó.

- Khó tìm ra tỷ lệ lỗi tối ưu và tỷ lệ lỗi mong muốn hợp lý. Giả sử bạn đã có một hệ thống giới thiệu sách đang hoạt động khá tốt. Làm thế nào để bạn biết mình có thể cải thiện nó thêm bao nhiêu nếu không có giải pháp cấp con người?

34. Cách xác định chất lượng mức con người

Giả sử bạn đang làm việc trên một ứng dụng hình ảnh y tế mà tự động đưa ra chẩn đoán từ ảnh X quang. Với tác vụ chuẩn đoán này, một người bình thường không có nền tảng y học nào ngoài một số đào tạo cơ bản có mức 15% lỗi. Một bác sĩ trẻ mới ra trường có mức 10% lỗi. Một bác sĩ dày dặn kinh nghiệm đạt được mức 5% lỗi. Và một nhóm nhỏ các bác sĩ cùng trao đổi và thảo luận từng ảnh đạt được mức 2% lỗi. Vậy cái nào trong những tỉ lệ lỗi này được định nghĩa là "chất lượng mức con người"?

Trong trường hợp này, tôi sẽ lấy 2% làm "chất lượng mức con người" cho tỉ lệ lỗi tối ưu của chúng ta. Bạn cũng có thể đặt 2% làm mức chất lượng mong muốn vì nó thỏa mãn cả ba lý do (trình bày ở chương trước) để so sánh với chất lượng mức con người:

- **Dễ dàng lấy dữ liệu từ người gán nhãn.** Bạn có thể nhờ một nhóm bác sĩ dán nhãn cho bạn với tỉ lệ lỗi 2%.
- **Phân tích lỗi dựa vào trực giác.** Bằng cách thảo luận hình ảnh với một nhóm các bác sĩ, bạn có thể dựa vào trực giác của họ để phân tích lỗi.
- **Dùng chất lượng mức con người để ước tính tỉ lệ lỗi tối ưu cũng như đặt ra "tỉ lệ lỗi mong muốn" khả thi.** Việc dùng mức 2% làm ước lượng về tỉ lệ lỗi tối ưu là hợp lý. Tỉ lệ lỗi tối ưu thậm chí có thể thấp hơn 2%, nhưng không thể lớn hơn, vì một nhóm bác sĩ có thể đạt được mức 2% lỗi. Ngược lại, sẽ không hợp lý khi sử dụng 5% hoặc 10% làm ước lượng cho tỉ lệ lỗi tối ưu, vì chúng ta biết các mức ước tính này quá lớn.

Khi lấy dữ liệu được gán nhãn, có thể bạn không muốn thảo luận về mọi bức ảnh với toàn bộ đội ngũ bác sĩ vì thời gian của họ rất đáng giá. Nhưng bạn có thể nhờ một bác sĩ trẻ mới ra trường gán nhãn cho phần lớn các trường hợp và chỉ dành những trường hợp khó hơn cho các bác sĩ có kinh nghiệm hơn hoặc cho đội ngũ bác sĩ.

Nếu hệ thống hiện tại của bạn có mức 40% lỗi, thì việc nhờ một bác sĩ mới ra trường (10% lỗi) hay một bác sĩ có kinh nghiệm (5% lỗi) để gán nhãn và đưa ra những phán đoán trực giác không đem lại nhiều khác biệt. Nhưng nếu hệ thống của bạn đang có 10% lỗi, thì việc xác định chất lượng mức con người ở mức 2% sẽ cho bạn các công cụ tốt hơn để tiếp tục cải thiện hệ thống của mình.

35. Vượt qua chất lượng mức con người

Bạn đang làm về nhận dạng giọng nói và có một tập dữ liệu là các đoạn âm thanh. Giả sử, tập dữ liệu đó có nhiều đoạn âm thanh nhiễu mà thậm chí con người cũng mắc phải 10% lỗi. Giả sử, hệ thống của bạn đã đạt được 8% lỗi. Liệu bạn có thể sử dụng bất kỳ kỹ thuật nào trong ba kỹ thuật được mô tả trong Chương 33 để tiếp tục tiến bộ nhanh chóng không?

Nếu có thể xác định một tập dữ liệu con mà con người đạt chất lượng cao hơn đáng kể so với hệ thống của bạn, thì bạn vẫn có thể sử dụng các kỹ thuật đó để thúc đẩy tiến trình nhanh chóng. Ví dụ, giả sử hệ thống của bạn tốt hơn nhiều so với con người trong việc nhận dạng giọng nói trong âm thanh nhiễu, nhưng con người vẫn tốt hơn trong việc ghi lại lời nói rất nhanh.

Đối với tập dữ liệu con với lời nói nhanh, bạn có thể:

1. lấy bản ghi thoại từ con người với chất lượng cao hơn so với đầu ra thuật toán của bạn.
2. dựa vào trực giác để hiểu lý do tại sao họ nghe chính xác một phát ngôn nhanh khi hệ thống của bạn chưa thể.
3. dùng chất lượng mức con người trên lời nói nhanh như một mục tiêu chất lượng mong muốn.

Tổng quát hơn, miễn là có các mẫu trong tập phát triển mà con người làm đúng và thuật toán của bạn làm sai, thì rất nhiều kỹ thuật được mô tả trước đây sẽ áp dụng được. Điều này vẫn đúng ngay cả khi chất lượng thật toán của bạn (tính trung bình trên toàn bộ tập phát triển/kiểm tra) đã vượt qua chất lượng mức con người.

Có nhiều ứng dụng học máy quan trọng mà máy đã vượt qua chất lượng mức con người. Ví dụ, máy làm tốt hơn trong việc dự đoán xếp hạng phim, ước lượng thời gian di chuyển của một chiếc xe giao hàng hoặc có chấp nhận hồ sơ vay vốn hay không. Chỉ một phần những kỹ thuật này là áp dụng được một khi con người còn gặp khó khăn trong việc xác định các mẫu mà thuật toán còn rõ ràng đang làm sai. Do đó, tiến độ thường chậm hơn trong các vấn đề mà máy đã vượt qua chất lượng mức con người, và ngược lại, nhanh hơn khi máy vẫn đang cố gắng bắt kịp con người.